模式识别
——模式识别与机器学习

刘文黎

liu_wenli@hust.edu.cn

土木与水利工程学院

第4章 非线性分类器

本章主要内容

4.1 引言
4.2 异或问题
4.3 两层感知器
4.4 多层感知器神经网络
4.5 反向传播算法*
4.6 代价函数的选择
4.7 神经网络大小的选择
4.8径向基函数*
4.9支持向量机：非线性情形*
4.10 决策树*
4.11 组合分类器
4.12 类别不平衡问题

4.1 引言

- 线性判别函数
  - 简单、实用、经济
  - 但线性不可分时错误率可能较大

线性问题不可分
- 噪声影响
- 问题本身
  - **采用非线性分类器**
  - 改变特征，使线性可分
    - **新特征**
    - 非线性变换

- 非线性判别函数
  - 各种函数的集合
  - 分段线性分类器、二次判别函数、多层感知器、反向传播算法、径向基函数、支持向量机、决策树、组合分类器等

分段线性判别函数

（详见教材p123 4.13线性二分分类器中l维空间的容量）

- 非线性函数可以用多段线性函数来逼近

  思路：

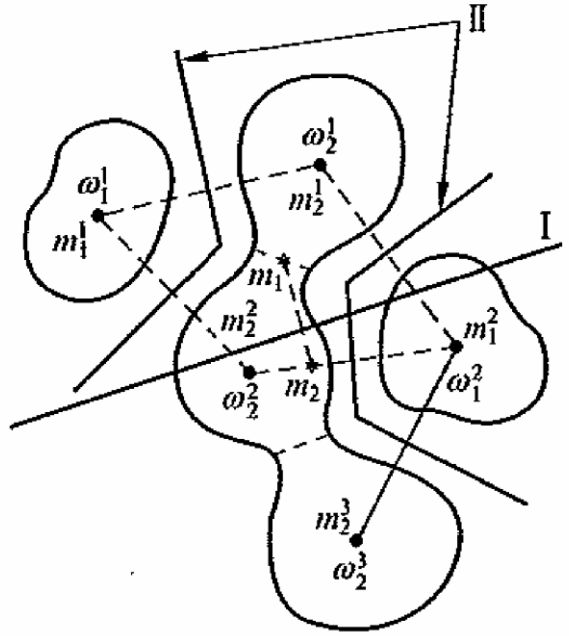
  如果两类可以划分为线性可分的若干子类，则可以设计多个线性分类器，实现分段线性分类器

- 分段线性判别函数：

  用多个线性分类器片段来实现非线性分类

- 联系：

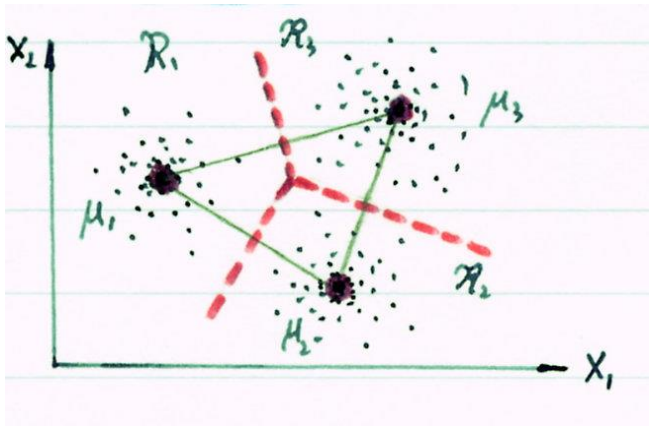
  上一章的多类线性判别函数实际就是分段线性判别函数

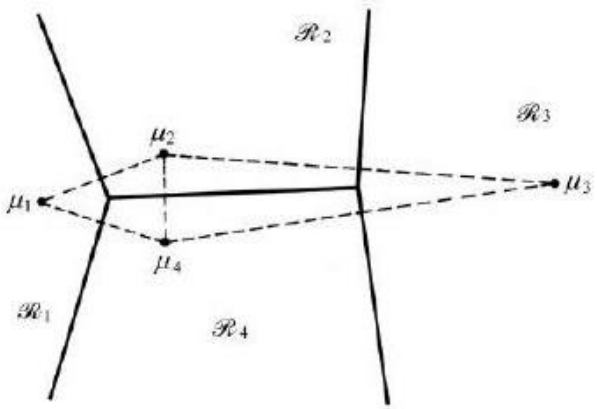


分段线性距离分类器示例

- 最简单情况：

  直接依据样本到各子类中心的距离判别

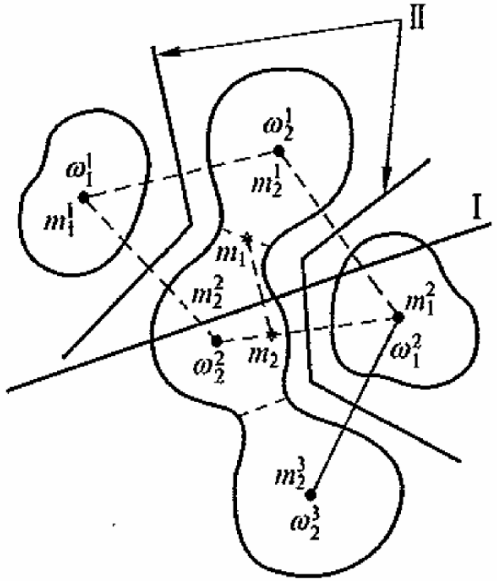




(b) 多类情况

- 更一般情况：

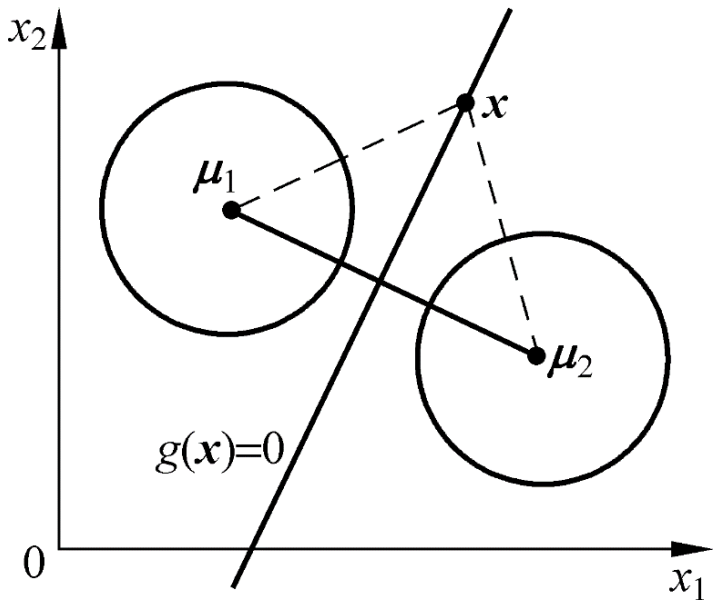
  对每个子类求一个线性判别函数



分段线性距离分类器示例

分段线性距离分类器

- 最小距离分类器：
  - 有$\omega_1, \cdots, \omega_c$个类，均值是$\boldsymbol{\mu_i}(i = 1, \cdots, c)$
  - 对样本x，如果$\|\boldsymbol{x} - \boldsymbol{\mu}_k\|^2 = \min\limits_{i=1,\cdots,c} \|\boldsymbol{x} - \boldsymbol{\mu}_i\|^2$，则决策$\boldsymbol{x}$属于$\omega_k$类
- 两类情况下，最小距离分类器就是两类的垂直平分面（超平面）



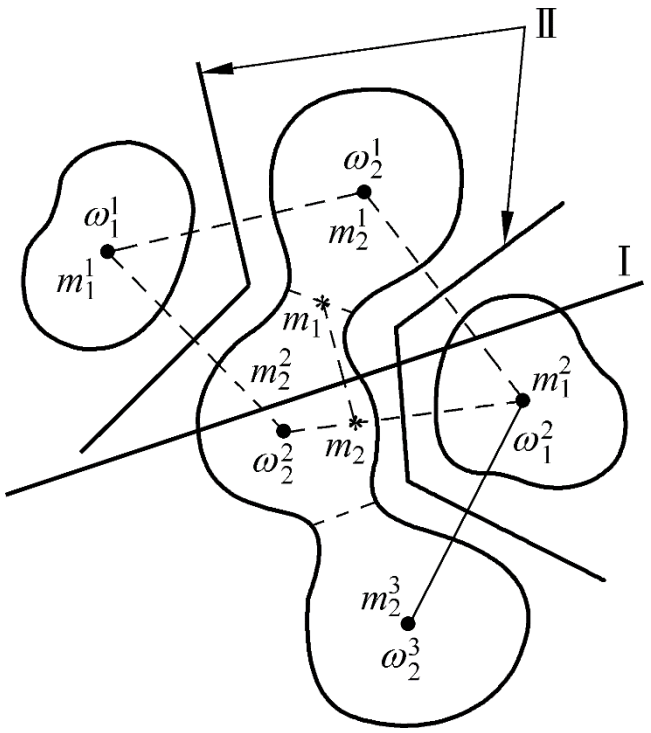
- 分段线性距离分类器：
  - 对一个待分类样本，把它分到距离最近的子类所属于的类
  - 把属于$\omega_i, i = 1,2,\cdots,c$类的样本区域$R_\mathrm{i}$划分
  - 为l_i个子区域$R_i^l, l = 1,2,\cdots,l$，每个子类的均值$\boldsymbol{m}_i^l$，

    对样本$\boldsymbol{x}, \omega_i$类的判别函数定义为

$$g_i(\boldsymbol{x}) = \min_{l=1,\cdots,l_i} \left\| \boldsymbol{x} - \boldsymbol{m}_i^l \right\|$$

  - 决策规则：

$$\text{若} g_k(\boldsymbol{x}) = \min_{i=1,\cdots,c} g_i(\boldsymbol{x})\text{，则决策} x \in \omega_k$$



一般的分段线性判别函数

- 对每个子类建立一般形式的线性判别函数：
  - 把每个类别划分成l_i个子类

$$\omega_i = \left\{\omega_i^1, \omega_i^2, \cdots, \omega_i^{l_i}\right\}, i = 1,2, \cdots, c$$

  - 对每个子类定义线性判别函数

$$g_i^l(\boldsymbol{x}) = \boldsymbol{w}_i^l \cdot \boldsymbol{x} + \omega_{i0}^l, l = 1, \cdots, l_i, i = 1, \cdots, c$$

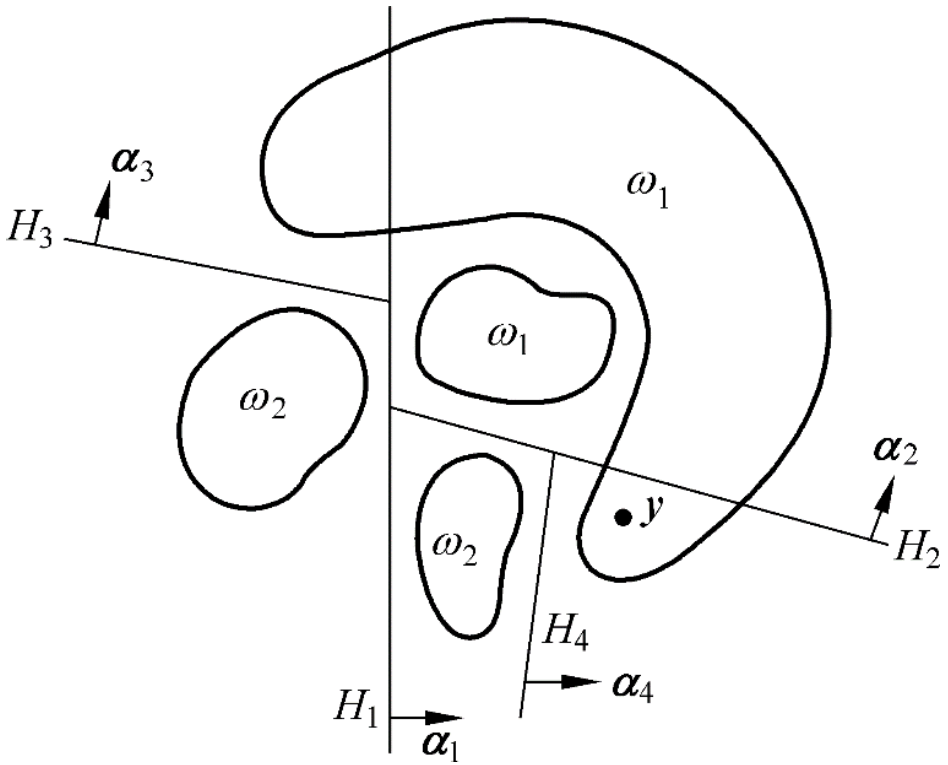
  - 类ω_i的分段线性判别函数就定义为

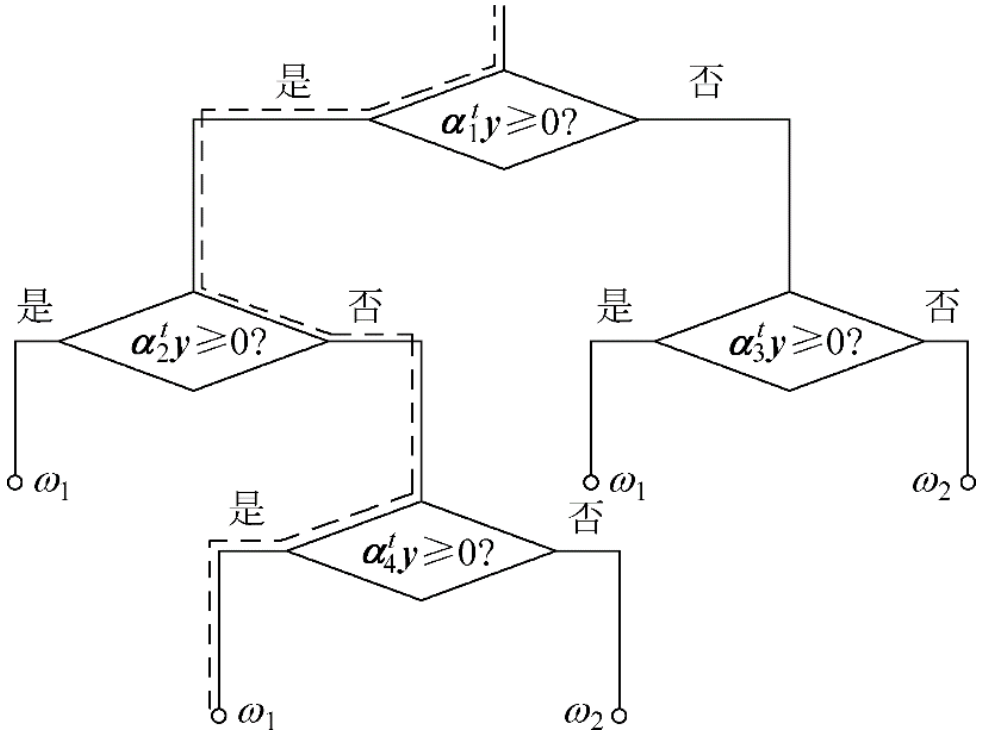
$$g_i(\boldsymbol{x}) = \max_{l=1,\cdots,l_i} g_i^l(\boldsymbol{x}), i = 1, \cdots, c$$

  - 决策规则：

$$\text{若} g_k(\boldsymbol{x}) = \max_{i=1,\cdots,c} g_i(\boldsymbol{x})\text{，则决策} x \in \omega_k$$

- 子类划分的三种设计方法：
  （1）人工确定子类的划分方案：多类分类器投票
  （2）已知或可假定子类数目，但不知道子类的划分：边划分边分类（下页）
  （3）子类数目无法事先确定：树状分段线性分类器





（2）已知或可以假定子类数目，但不知道子类的划分：

ω_i，$i=1,\cdots,c$，ω_i类中有l_i个子类，$\alpha_i^l(k)$为子类权值。

(a) 初始化权值，$\boldsymbol{\alpha}_i^l(0)$，$i=1,\cdots,c$，$l=1,\cdots,l_i$

(b) 对某个样本$\mathbf{y}_k \in \omega_j$，找出$\omega_j$类的子类中最大的判别函数

$$\boldsymbol{\alpha}_j^m(t)^T y_k = \max_{l=1,\cdots,l_j}\{\boldsymbol{\alpha}_j^l(t)^T \mathbf{y}_k\}$$

① 若$\boldsymbol{\alpha}_j^m(t)^T \mathbf{y}_k > \boldsymbol{\alpha}_i^l(t)^T \mathbf{y}_k$，$\forall i=1,\cdots,c$，$i\neq j$，$l=1,\cdots,l_i$, 则$\boldsymbol{\alpha}_i^l(t)$不变

② 若对某个$i\neq j$，存在子类l，有$\boldsymbol{\alpha}_j^m(t)^T \boldsymbol{y}_k \leq \boldsymbol{\alpha}_i^n(t)^T \mathbf{y}_k$，则$\mathbf{y}_k$被错分，对其中最大者（记为$i,n$），修正：

$$\boldsymbol{\alpha}_j^m(t+1) = \boldsymbol{\alpha}_j^m(t) + \rho_k \mathbf{y}_k$$

$$\boldsymbol{\alpha}_i^n(t+1) = \boldsymbol{\alpha}_i^n(t) - \rho_k \mathbf{y}_k$$

(c) 对下一个样本重复 (b)，直到收敛

二次判别函数

- 一种常用的固定函数类型的分类方法

二次判别函数一般形式

$g(x) = x^T W x + w^T x + w_0$　需要*d(d+3)/2+1*个参数，计算复杂

在应用时采用参数化的方法，定义数据正态分布：

$$g_i(x) = k_i^2 - (x - \hat{m}_i)^T \hat{\sum}_i^{-1} (x - \hat{m}_i), \quad i = 1,2$$

$$\hat{m}_i = \frac{1}{N_i} \sum_{j=1}^{N_i} x_j$$

$$\hat{\Sigma}_i = \frac{1}{N_i - 1} \sum_{j=1}^{N_i} (x_j - \hat{m}_i)(x_j - \hat{m}_i)^T$$

- 适用于两类分布较成团的情况，k_i阈值，控制决策椭球大小

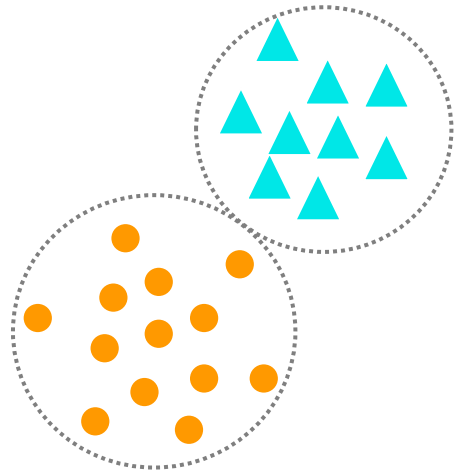
- 第一种情况

假设每一类样本都满足高斯分布，如果两类样本分布各自成团且都比较集中，则可以分别定义两类的判别函数为：

$$g_i(x) = k_i^2 - (x - \hat{m}_i)^T \hat{\sum}_i^{-1} (x - \hat{m}_i), \quad i = 1,2$$

$$\hat{m}_i = \frac{1}{N_i} \sum_{j=1}^{N_i} x_j$$

$$\hat{\Sigma}_i = \frac{1}{N_i - 1} \sum_{j=1}^{N_i} (x_j - \hat{m}_i)(x_j - \hat{m}_i)^T$$



判别函数：$g(x) = g_1(x) - g_2(x)$

决策规则表示为：$g(x) \begin{matrix} > \\ < \end{matrix} 0 \quad \Rightarrow \quad x \in \begin{cases} \omega_1 \\ \omega_2 \end{cases}$

若决策出现错误，可以通过调整两类的阈值来减小错误率。

- 第二种情况

假设其中一类样本分布比较接近高斯分布（分布呈团状），而另外一类较均匀的分布在第一类周围的两类问题，可以采用下述方法构造一个判别函数：

只需要计算第一类（高斯分布）样本的均值和样本协方差矩阵。

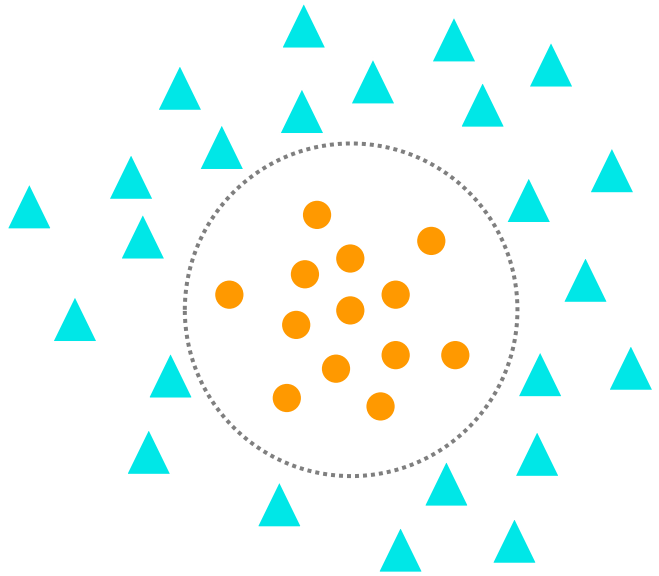
$$\hat{m}_1 = \frac{1}{N_1}\sum_{j=1}^{N_1} x_j \qquad \hat{\Sigma}_1 = \frac{1}{N_1 - 1}\sum_{j=1}^{N_1}(x_j - \hat{m}_1)(x_j - \hat{m}_1)^T$$

定义二次判别函数：$g(x) = k^2 - (x - \hat{m}_1)^T \hat{\Sigma}_1^{-1} (x - \hat{m}_1)$

决策规则表示为：$g(x) \gtrless 0 \quad \Rightarrow \quad x \in \begin{cases} \omega_1 \\ \omega_2 \end{cases}$

$g(x) = 0$在X的空间确定了一个超椭球面

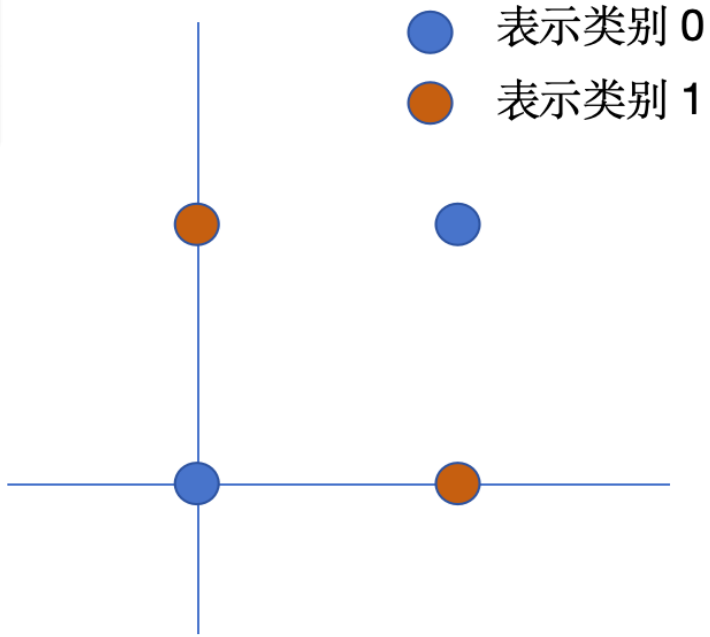
同样可以调整阈值k^2来调整分类错误率。



4.2 异或问题

在逻辑运算中，XOR（Exclusive OR，异或）是一个二进制操作。它遵循以下规则：当两个输入中恰好有一个为真（1）时，输出为真（1）；否则输出为假（0）。XOR 的逻辑表如下：

输入x1	输入x2	XOR输出
0	0	0
1	0	1
0	1	1
1	1	0



XOR 问题最大的特征是它是线性不可分的，

不能通过一条直线（或平面）将输入空间中的不同类别完全分开

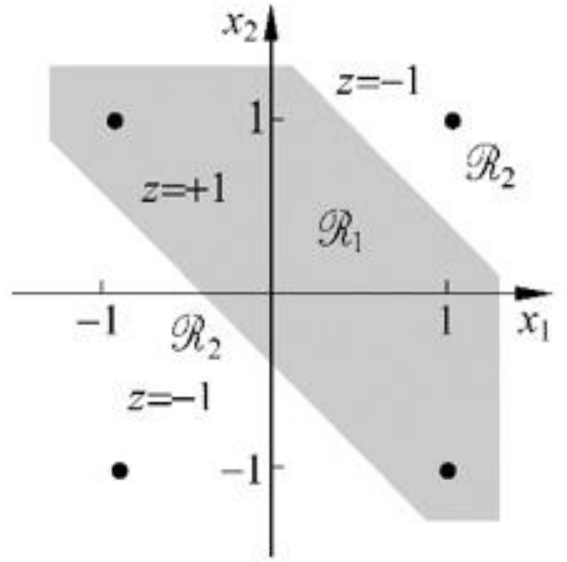
4.3 两层感知器

- 单个感知器神经元无法解决非线性问题

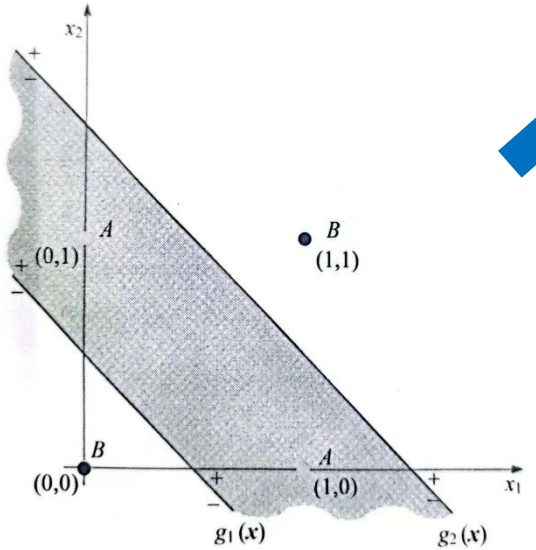
  异或（XOR）问题

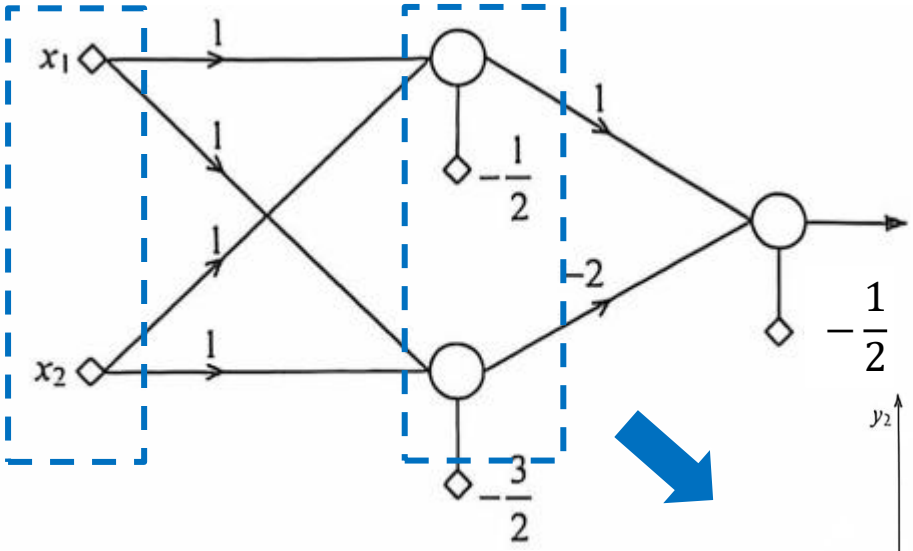
  若（1,1）点和（-1，-1）点同属于第一类，而（1，-1）点和（-1，1）点同属于第二类

  图中给出了类在空间的位置，一条直线不能将这两类分开，即对于该分类问题，单个感知器神经网络无法正确分类

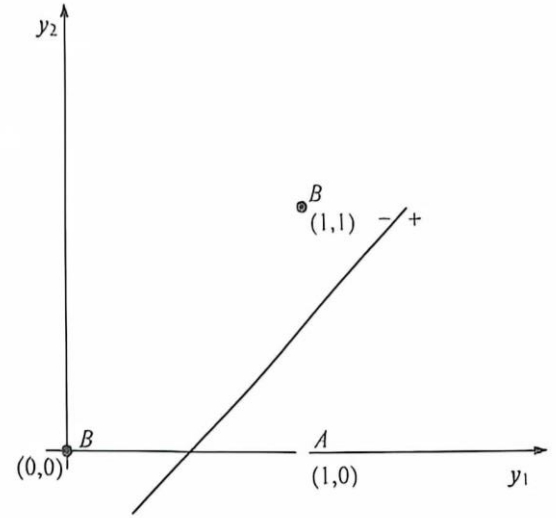


- 异或(XOR)问题必须用两层感知器实现





求解XOR问题的两层感知器



XOR问题的两个计算阶段真值表

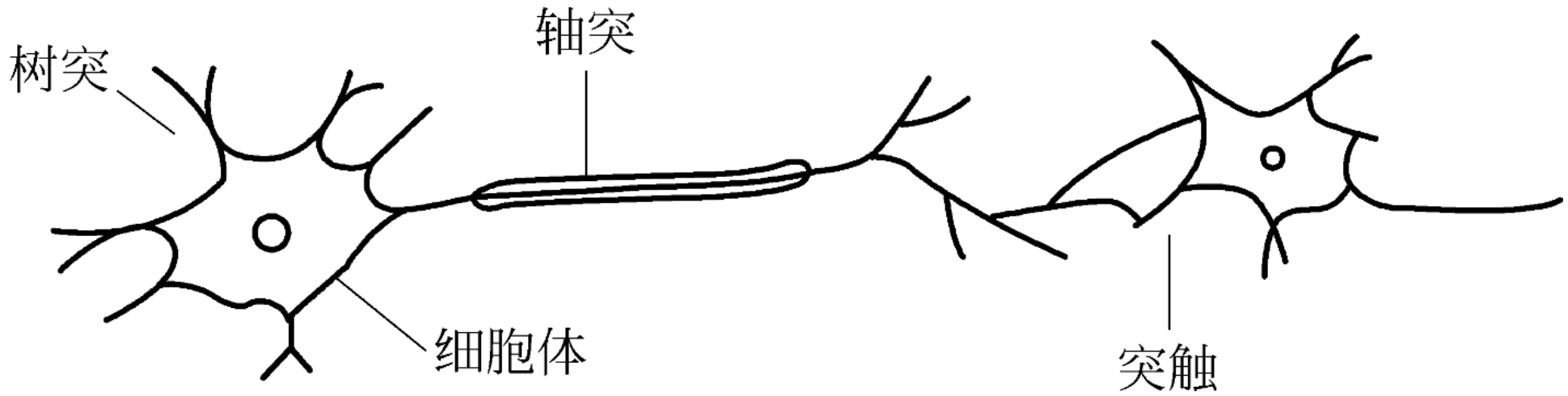
第一个阶段				第二个阶段
x_1	x_2	y_1	y_2	
0	0	0(−)	0(−)	$B(0)$
0	1	1(+)	0(−)	$A(1)$
1	0	1(+)	0(−)	$A(1)$
1	1	1(+)	1(+)	$B(0)$

由此生成的多层结构视为感知器的推广，称为**两层感知器或两层前馈神经网络**

4.4 多层感知器神经网络

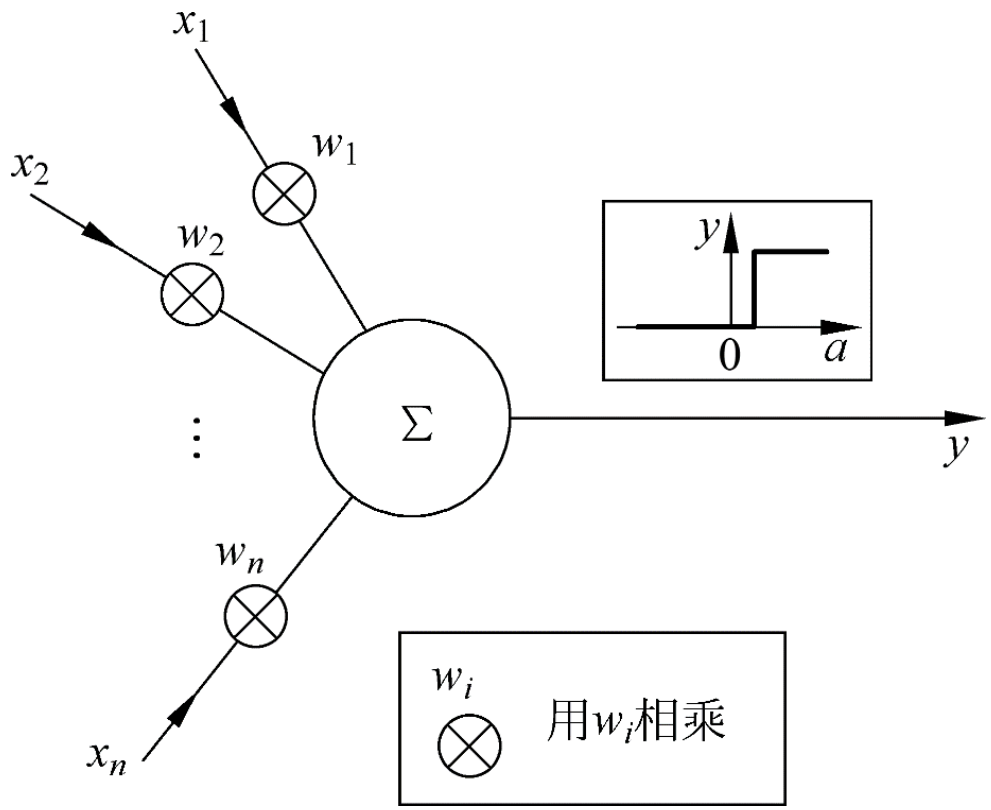
4.4.1 神经元与感知器

- 神经元（neuron）
  - 细胞体（cell body）、树突（dendrites）、轴突（axon）、突触（synapse）

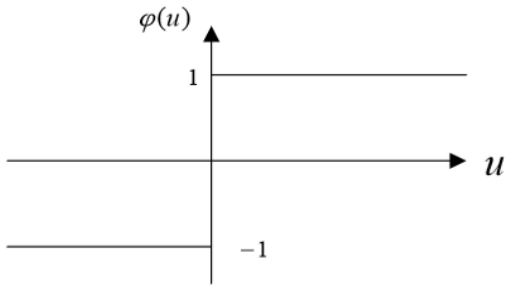


- 基本的神经元模型：
  - McCulloch-Pitts Model (1943)
  - 阈值逻辑单元(Threshold Logic Unit, TLU)

$$y = \theta\left(\sum_{i=1}^{n} w_i x_i + w_0\right)$$

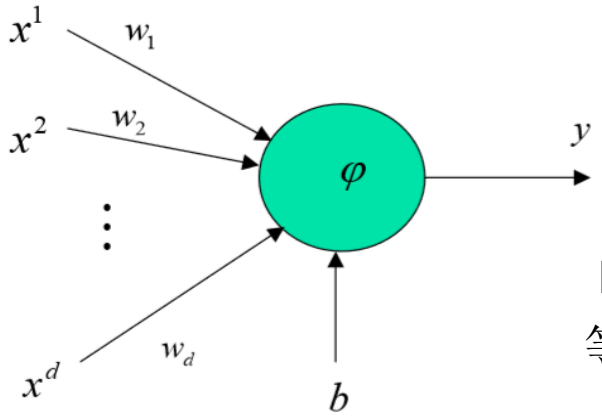


- 感知器（Perceptrons）模型



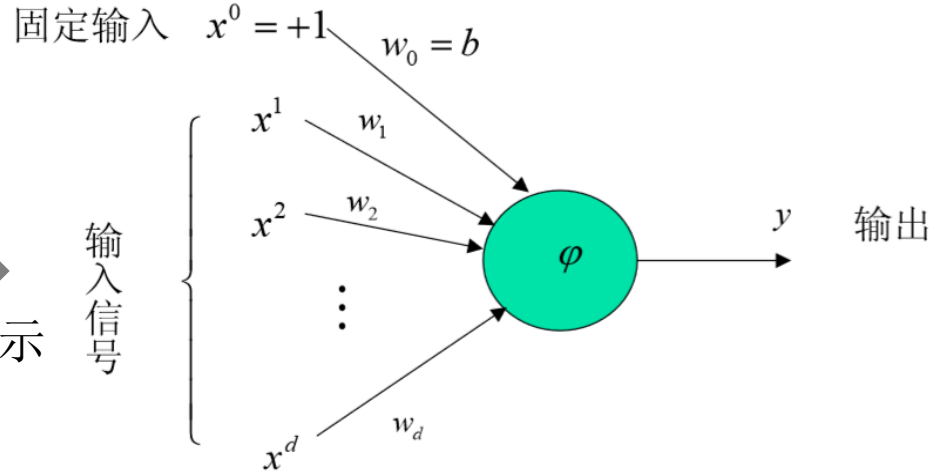
$$\varphi(u) = \mathrm{sgn}(u).$$

$$y = \mathrm{sgn}\left(\sum_{i=1}^{d} w_i x^i + b\right)$$



线性分类器！

等价表示



- 练习:计算图1、2中，输入值经过激活函数(阶跃函数) 后输出为多少?

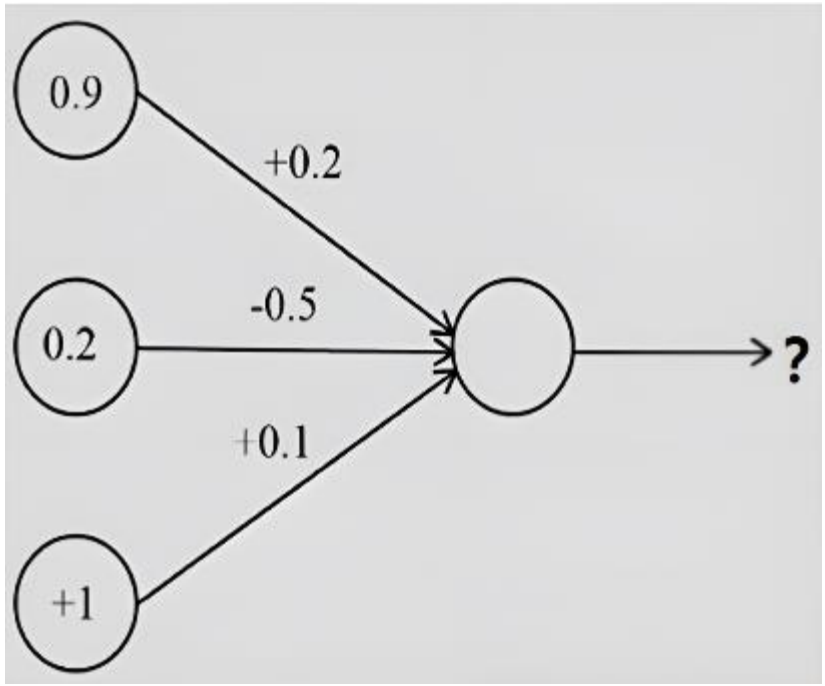


图1

+1

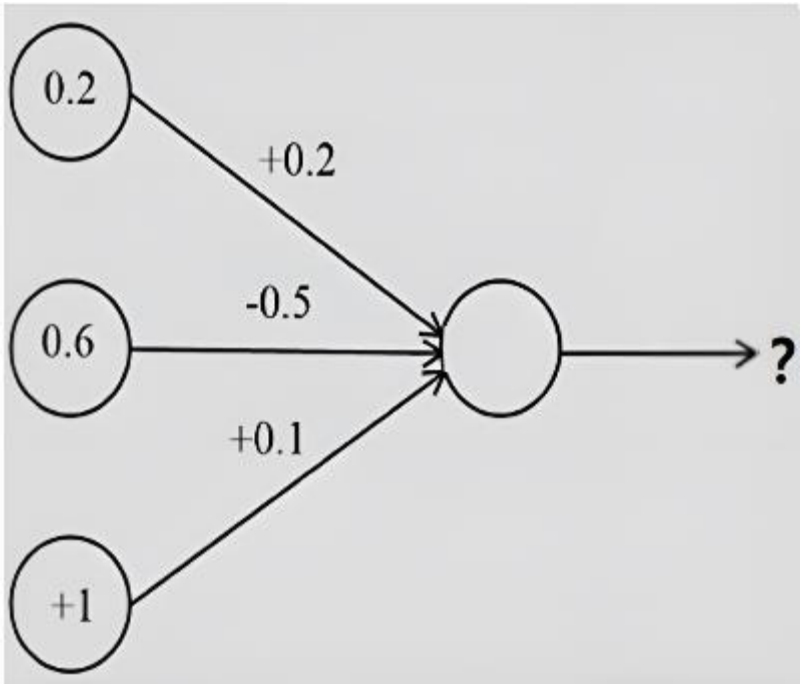
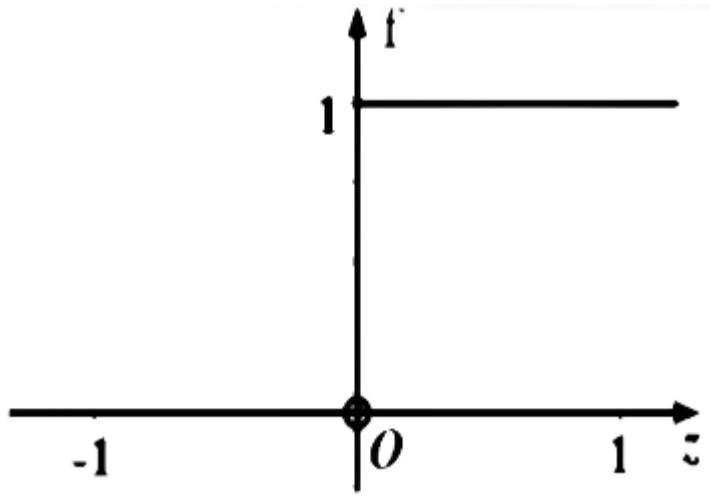


图2

0

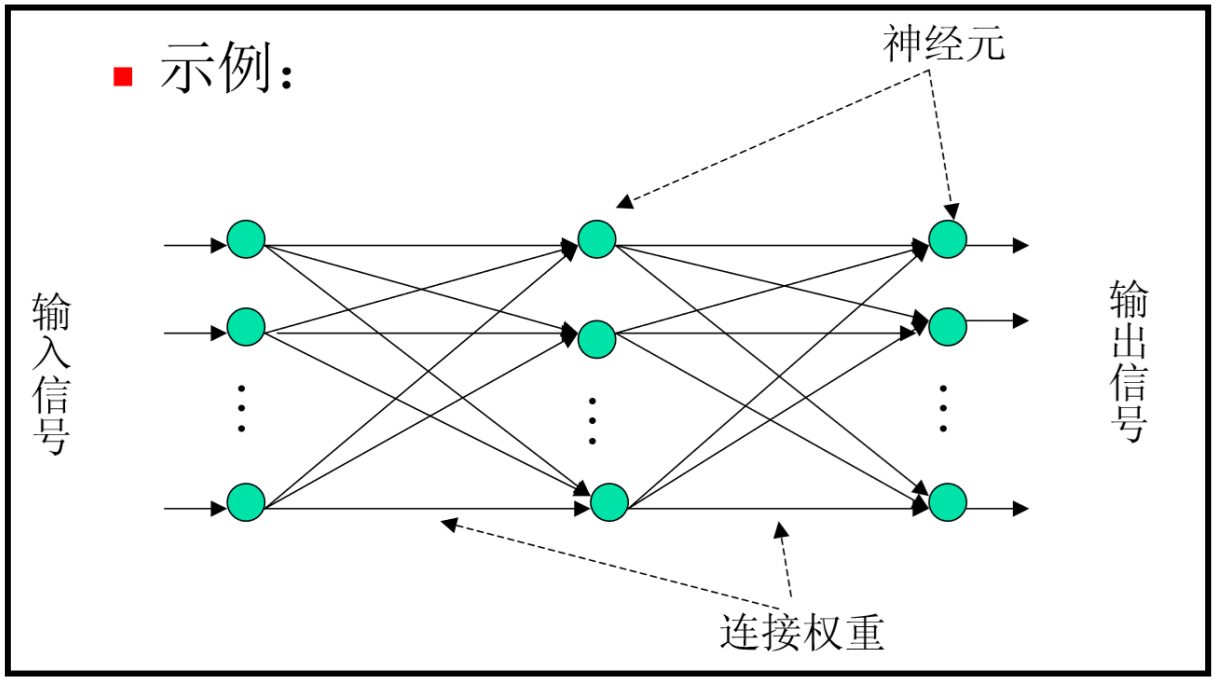


阶跃函数

4.4.2 神经网络的概念及发展

- （人工）神经网络的基本结构：

大量简单的计算单元（结点）以某种形式相连接，形成一个网络，其中的某些因素，如连接强度（权值）、结点计算特性甚至网络结构等，可依某种规则随外部数据进行适当的调整，最终实现某种功能



- 发展历程

- McCulloch-Pitts模型(McCulloch, Pitts 1943);
  心理学家w.McCulloch 和数组家W. Pitts合作提出的兴奋与抑制型神经元模型
- Hebb提出神经元学习准则(1949)
- 感知机(Rosenblatt1958);
- Hopfield模型(1982)
- 多层感知机与反向传播算法(Rumelhart, Hinton 1986).

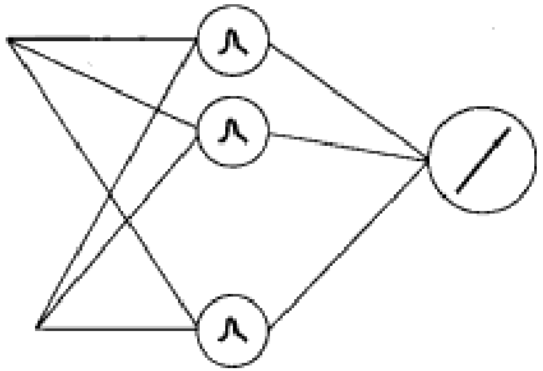


图 5-16　径向基函数神经网络示意图

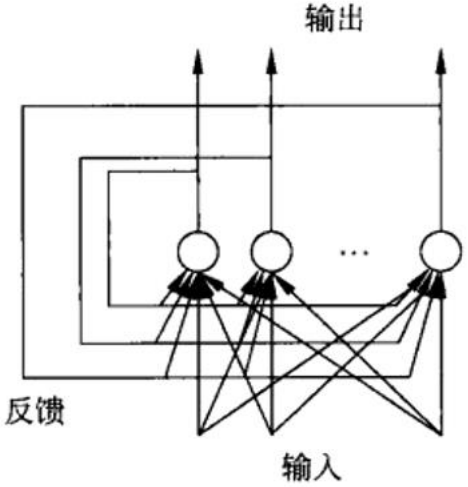
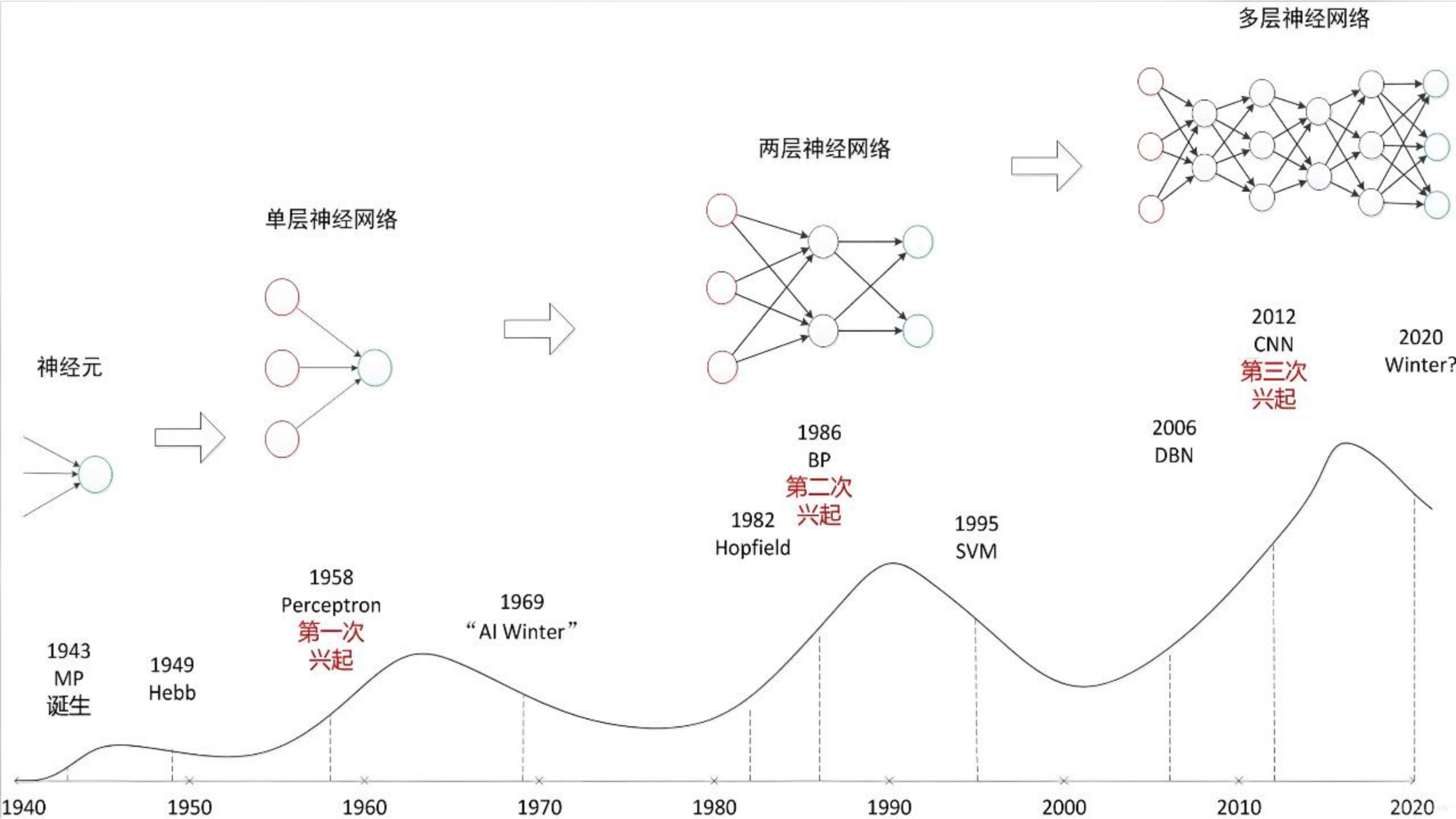


图 5-17　Hopfield 网络示意图


多层神经网络
两层神经网络
单层神经网络
神经元
2012
CNN
第三次
兴起
2020
Winter?
2006
DBN
1986
BP
第二次
兴起
1982
Hopfield
1995
SVM
1958
Perceptron
第一次
兴起
1969
“AI Winter”
1943
MP
诞生
1949
Hebb
1940
1950
1960
1970
1980
1990
2000
2010
2020

4.4.3 关于神经网络的问题

- 基本的神经元模型

$$y = \theta\left(\sum_{i=1}^{n} w_i x_i + w_0\right)$$

把神经元的两个可能的输出值看做两类，上式所描述的神经元的传递函数实际就是一个线性分类器

- **学习算法收敛性问题**

对线性不可分的数据，算法不收敛！

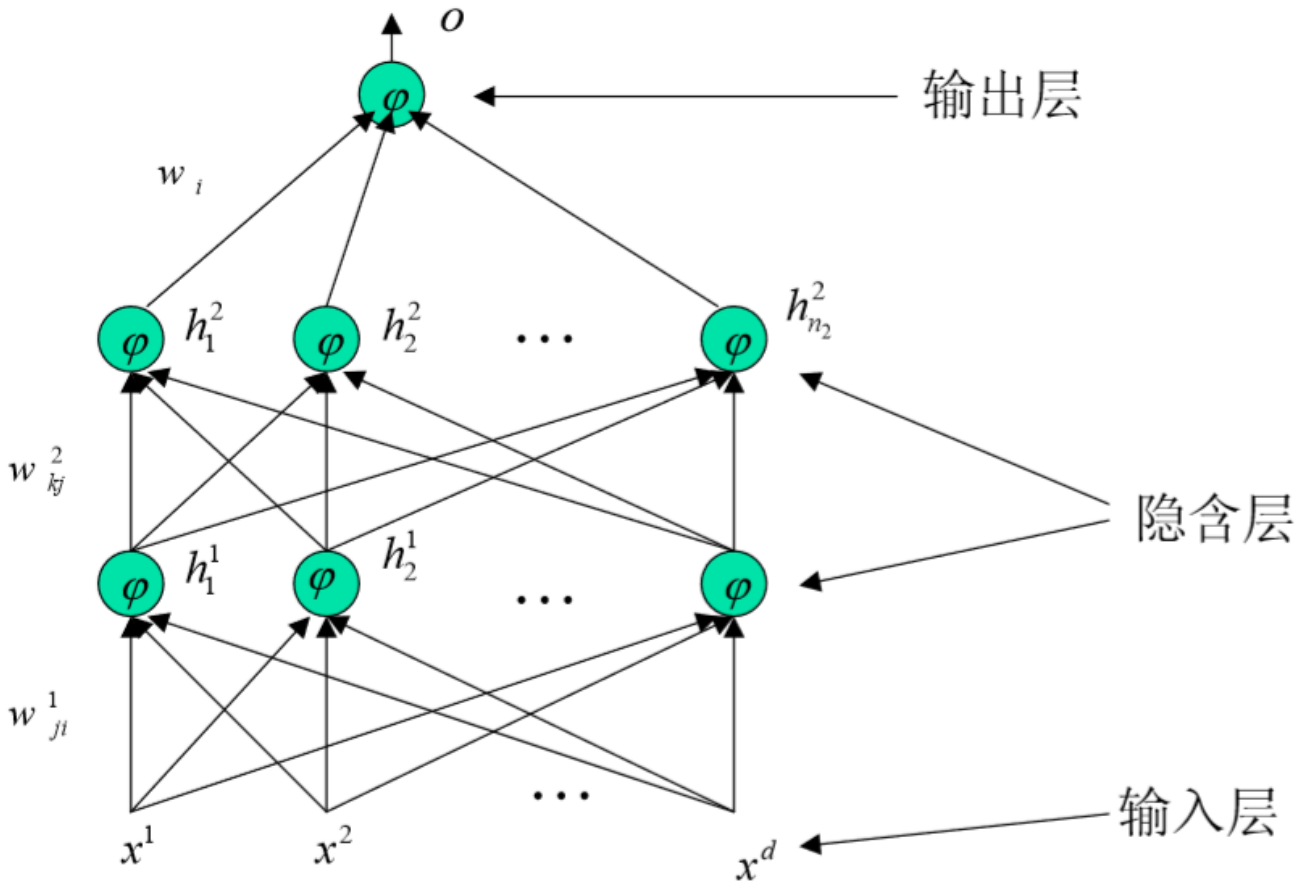
回顾上一章节

- **解决方案**

多层感知器，非线性分类器

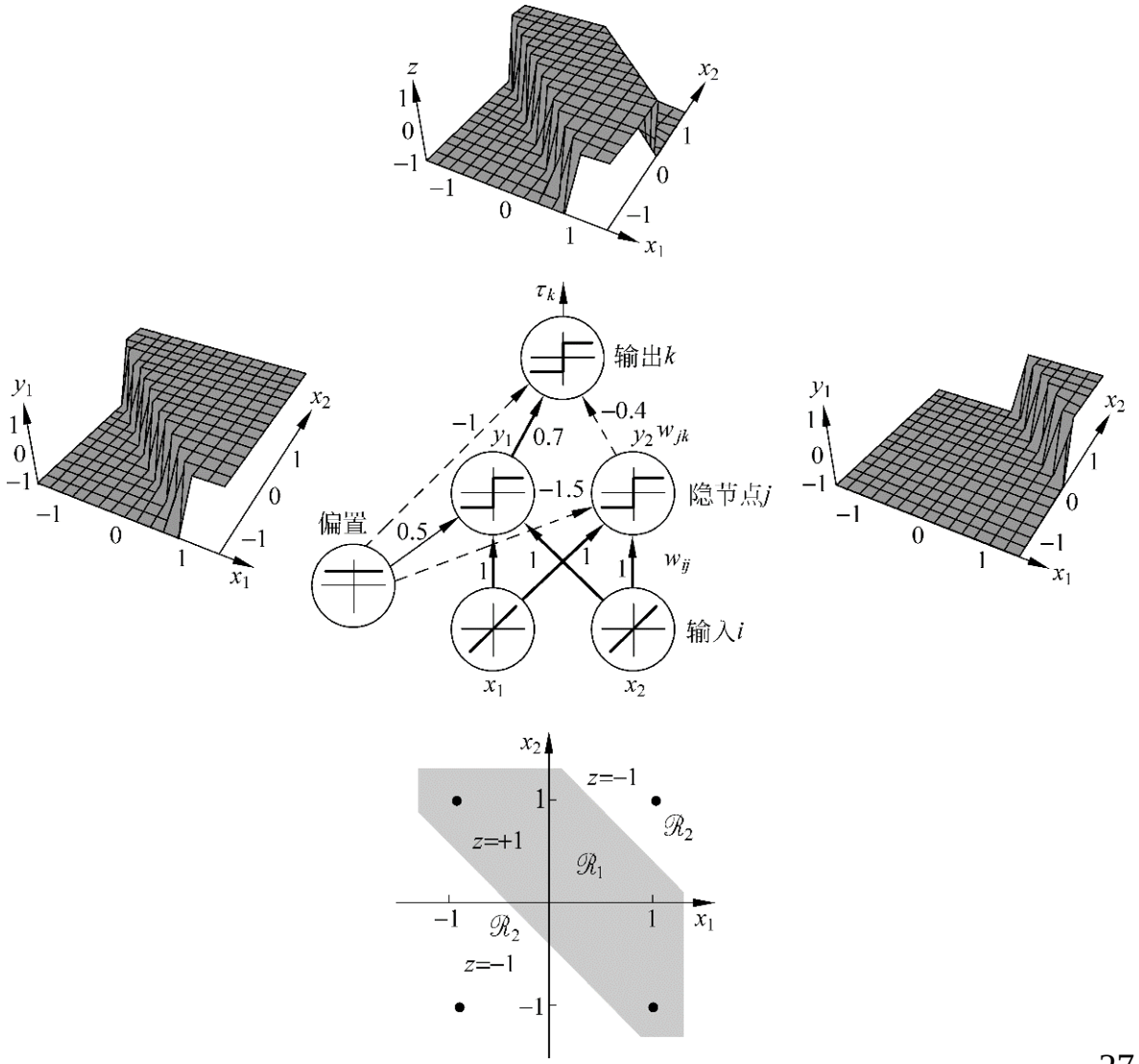
- 多层感知器（multi-layer perceptron — MLP）具有学习任意复杂的非线性映射的能力
- 多层感知器可看作一种通用的**非线性分类器设计方法**
- **三个要素**
  - 神经元的计算特性（传递函数）
  - 网络的结构（连接形式）
  - 学习规则

- 多层感知器（Multilayer Perceptrons）结构



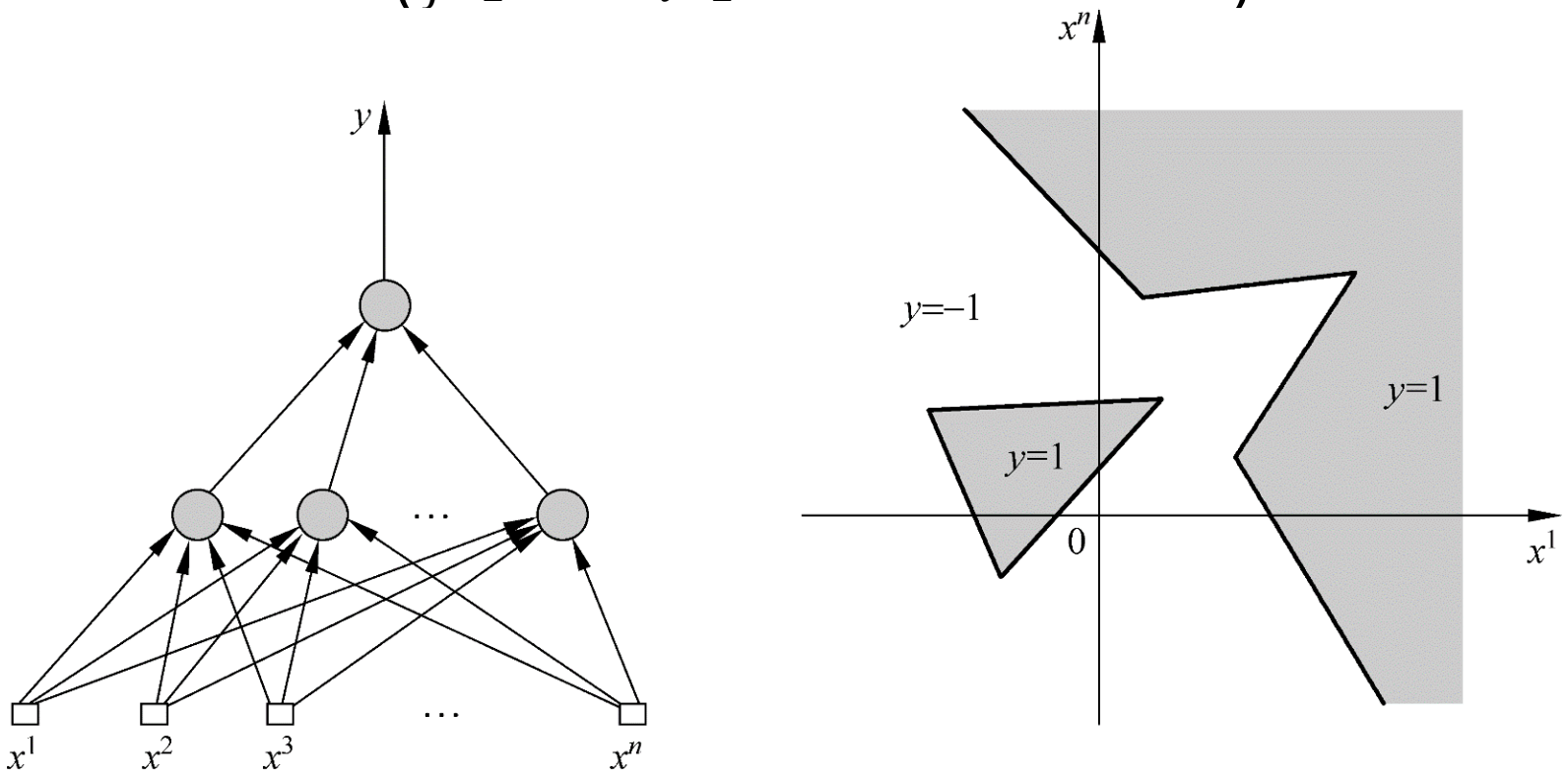
4.4.4 用多个感知器实现非线性分类

- 单个感知器神经元无法解决非线性问题
- 多个神经元分层组合实现复杂的空间形状分割



- 对于任意复杂形状的分类区域，总可以用多个神经元组成一定的层次结构来实现分类

$$y = \theta\left\{\sum_{j=1}^{n} v_j\,\theta\left(\sum_{i=1}^{m} w_{ij}x_i + w_{0j}\right) + v_0\right\}$$



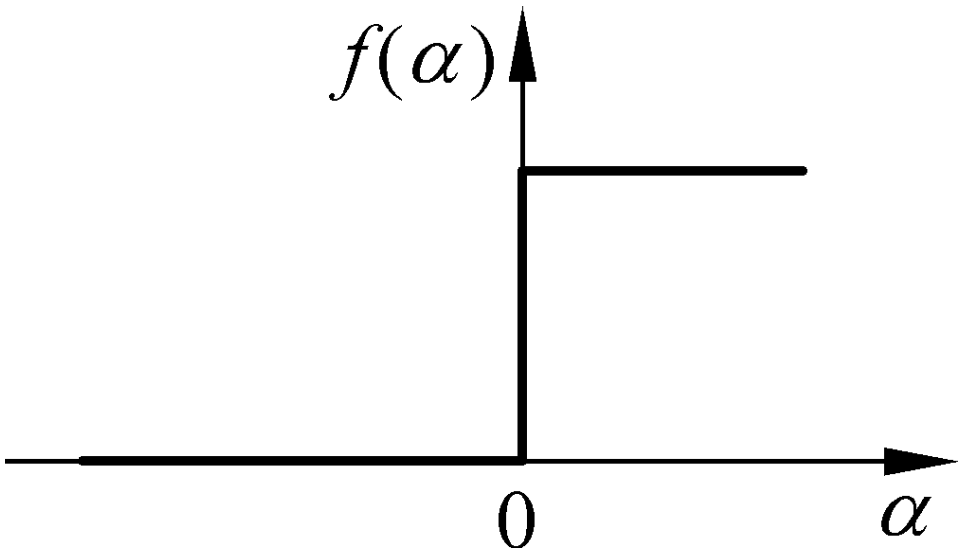
(a) 感知器模型　　(b) 输入空间中的两上区域

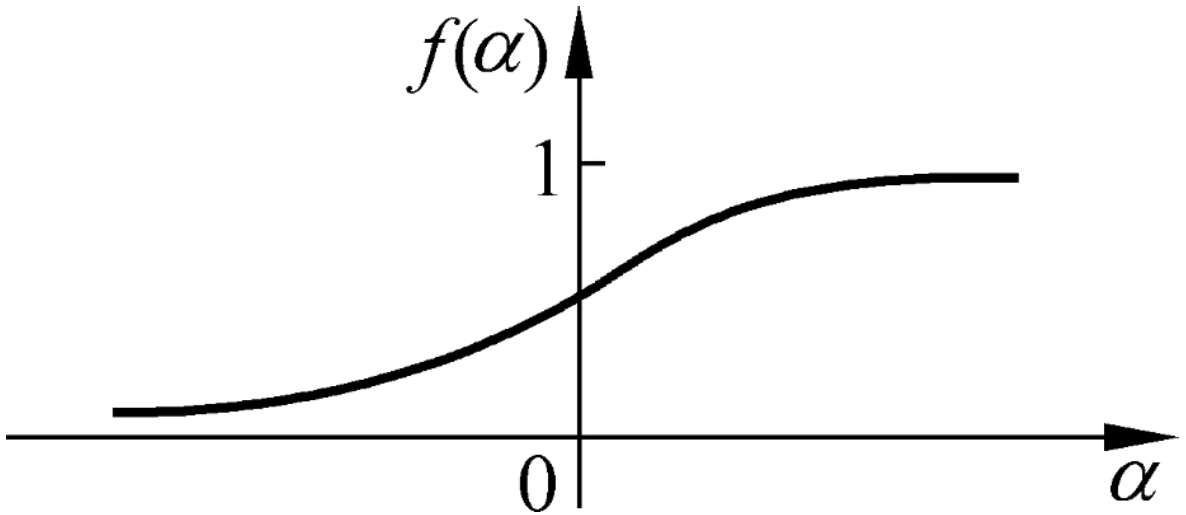
- 神经网络的输出取决于输入和连接的权值，其工作过程主要分两个阶段：
  - 学习过程—**通过不断地学习修改权值**
  - 工作过程—权值确定后，可**计算输出**
- 单层感知器可通过感知器算法进行学习，调整权值，完成线性分类，它的输入是训练集的样本，输出是期望值，对外是透明的。
- 多层感知器中的中间隐层学习算法，对外是不透明的，隐层权值调整有困难。在20世纪80年代提出了误差反向传播算法，来计算隐层的权值。

4.5 反向传播算法

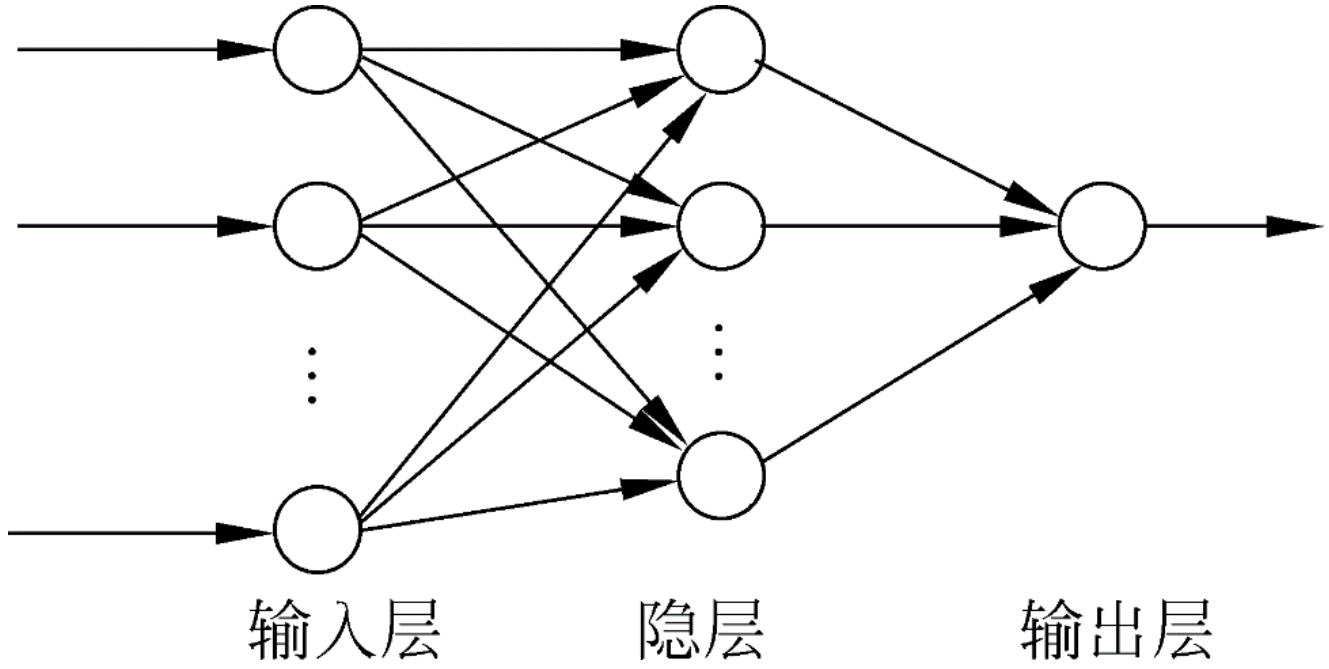
- 利用Sigmoid函数代替感知器中的阈值函数来构建神经网络：

$$f(\alpha) = \frac{1}{1 + e^{-\alpha}}$$



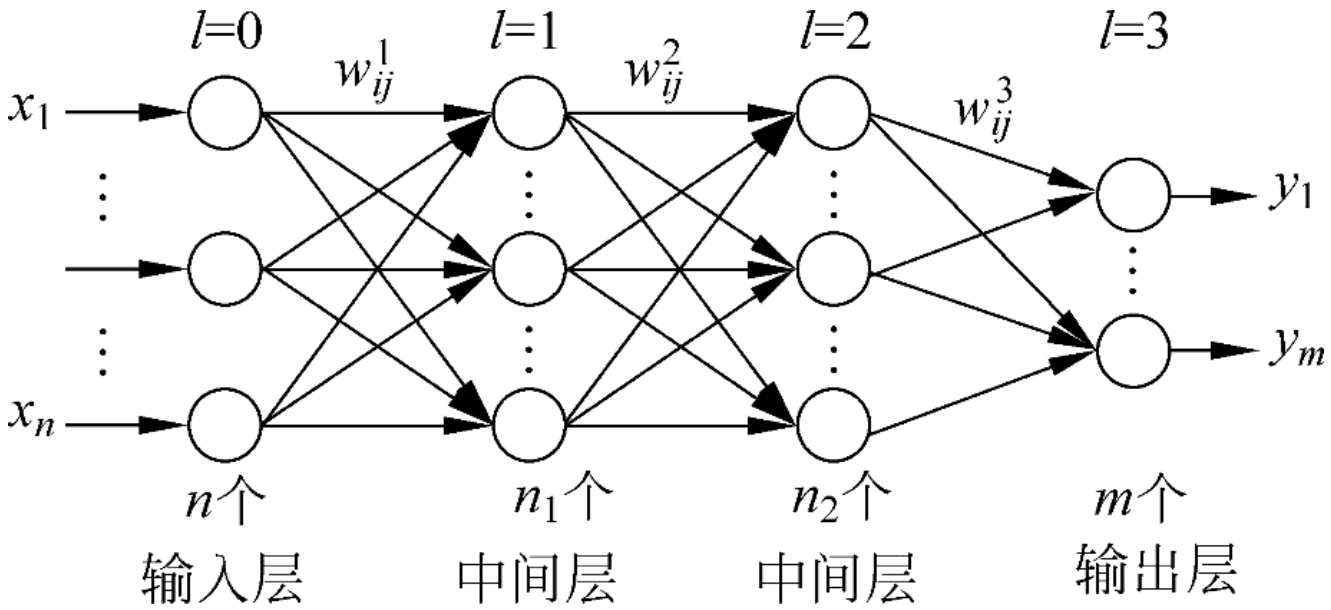


- 多层感知器（multi layer perceptron，MLP）网络：
  - 是一种可普遍适用的非线性学习机器，能够实现任意复杂的函数映射
  - 采用反向传播(Back Propagation)算法训练权值



- BP算法：

  目标函数是均方误差，采用梯度下降法通过调整各层的权值求目标函数最小化

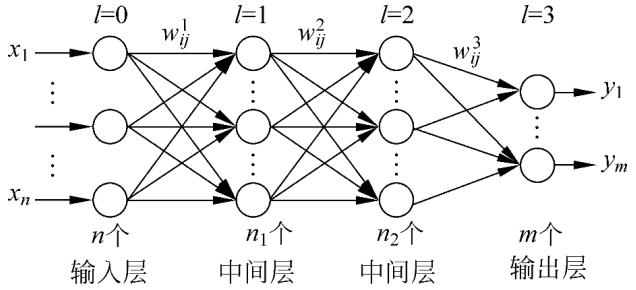


(1) 用小随机数进行权值初始化，设训练时间t=0

(2) 给出一个训练样本 $\boldsymbol{x} = [x_1, \cdots, x_n]^T \in R^n$ 和期望输出 $\boldsymbol{D} = [d_1, \cdots, d_m]^T \in R^m$

(3) 计算在x输入下的实际输出:

$$y_r = f(\sum_{s=1}^{n_{L-2}} \omega_{sr}^{l=L-1} \cdots f\left(\sum_{j=1}^{n_1} \omega_{jk}^{l=2} f(\sum_{i=1}^{n} \omega_{ij}^{l=1} x_i)\right), r = 1, \cdots, \mathrm{m}$$



(4) 从输出层开始调整权值：

对第l层，用下面的公式修正权值

$$\omega_{ij}^l(t+1) = \omega_{ij}^l(t) + \Delta\omega_{ij}^l(t), \qquad j = 1, \cdots, n_l, i = 1, \cdots, n_{l-1}$$

$$\Delta\omega_{ij}^l(t) = -\eta\delta_j^l x_i^{l-1}, \qquad \eta\text{为学习步长}$$

对输出层：$\delta_j^l = -y_j(1-y_j)(d_j - y_j), j = 1, \cdots, m$

对中间层：$\delta_j^l = x_j^l\left(1 - x_j^l\right)\sum_{k=1}^{n_{l+1}} \delta_k^{l+1} w_{jk}^{l+1}(t), j = 1, \cdots, n_l$

(5) 重新计算输出，考查误差指标。如达到终止条件则终止，

否则置$t = t + 1$，返回(2)

注意，上述BP算法是针对sigmoid传递函数：

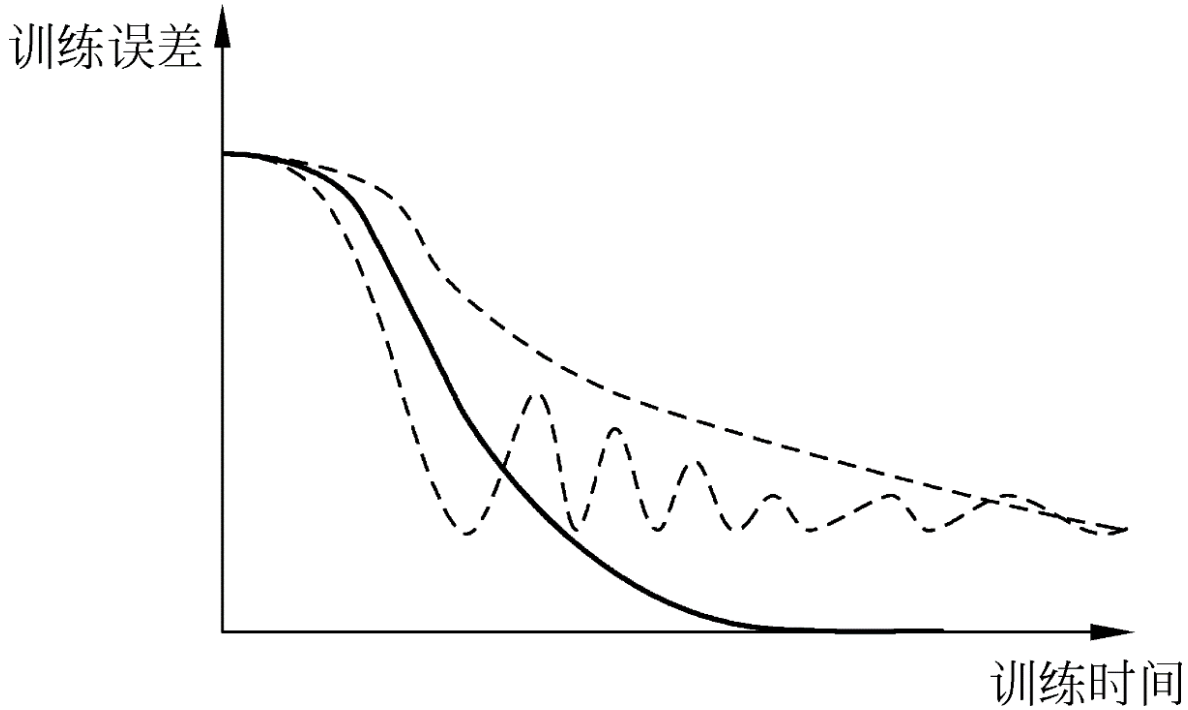
$$f(\alpha) = 1/(1 + e^{-\alpha}),$$

$$f'(\alpha) = f(\alpha)(1 - f(\alpha))$$

当改用其他传递函数时其梯度函数需要相应改变。

- 说明：
  - 算法可能收敛于局部极小点（梯度算法）
  - 与初值、步长等的选择有关
  - 更与网络结构（结点数目）有关

- 优化方式：
  - 多凭经验或试验选择
  - 如η时变，加记忆项，
    网络结点可剪裁，等等。



- BP算法的特点

  - BP算法解决了单层感知器难以处理的**非线性可分模式**的分类问题，广泛应用于模式识别和自动控制等领域
  - BP网络本质上是一种从输入到输出的映射方法，不需要输入与输出之间的精确数学表达式（与模型无关），只需用**已知的模式样本**对BP网络进行训练，网络便具备输入与输出之间的映射能力。
  - BP算法的关键在于**中间隐层**的学习规则，而中间隐层相当于对输入信息的特征抽取器

- BP算法的不足

– BP算法是一个**非线性优化问题**，存在**局部极小值问题**

– **收敛速度较慢**，通常需要迭代数千次或更多，一般仅适用于离线的模式识别任务

– BP网络是前馈网络，运行时为单向传播，没有反馈机制，其输入输出关系不是非线性动力学系统，而是**简单的映射**

– 隐层数和隐层的神经元个数的**选择缺乏理论指导**，多凭经验确定

– 新增样本可能会**影响到已完成学习的样本**，并要求各样本的特征数量保持一致

- BP算法的改进
  - 使用动力项，加快收敛速度

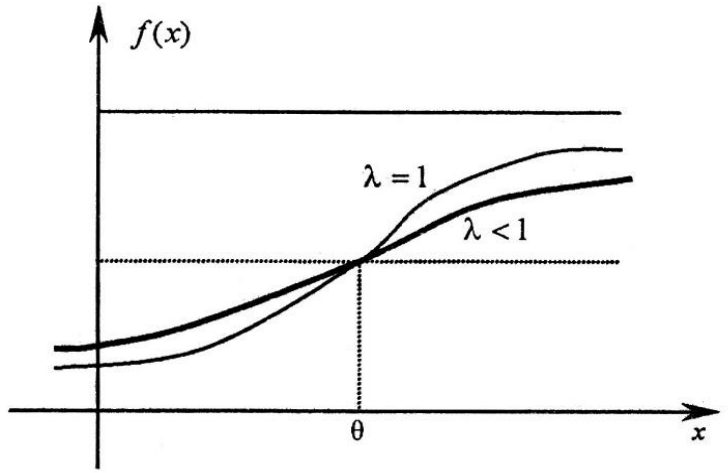
$$\boldsymbol{w_{ij}}(\boldsymbol{t}+1)=\boldsymbol{w_{ij}}(\boldsymbol{t})+\eta\delta_i\boldsymbol{O_j}+\alpha(\boldsymbol{w_{ij}}(\boldsymbol{t})-\boldsymbol{w_{ij}}(\boldsymbol{t}-1))$$

若误差下降，则取 $\alpha > 1$ (如动量系数$\alpha = 1.7$)

若误差上升，则取 $0 < \alpha < 1$ ($\alpha = 0.1 \sim 0.8$)

  - 修改激活函数

$$f(x)=\frac{1}{1+e^{-\lambda(x-\theta)}}$$ θ阈值



参数 λ 的影响

  - 学习步长η的自适应调整

$$\eta^{(t+1)}=\eta^{(t)}\cdot\frac{\boldsymbol{E}^{(t-1)}}{\boldsymbol{E}^{(t)}}$$

E样本平均误差(或准则函数J)，t 迭代次数

4.6 代价函数的选择

- 损失函数（Loss Function）：是定义在单个样本上的，是指一个样本的误差
- **代价函数（Cost Function）**：是定义在整个训练集上的，是所有样本误差的平均，也就是所有损失函数值的平均
- 目标函数（Object Function）：是指最终需要优化的函数，一般来说是经验风险+结构风险，也就是（代价函数+正则化项）

代价函数的选择直接决定了**模型优化的方向：**

例如，在回归问题中，**均方误差（MSE）**是一种常用的代价函数，它衡量了预测值与真实值之间的平方差。通过最小化MSE，我们可以使模型的预测结果更加接近真实值。

而在分类问题中，**交叉熵损失函数**则更为常见，它适用于处理具有概率分布的输出。因此，选择适当的代价函数可以使模型更加符合问题的特性，从而提高模型性能。

4.7 神经网络大小的选择

- 人工神经网络的三个要素：
  - 神经元的传递函数 — 比如选Sigmoid函数
  - 网络结构 — 通常三层（1个隐层）即可满足常见任务需求
  - 连接权值学习 — 通常采用 BP算法学习
- 过学习（over-fitting）与欠学习（under-fitting）问题
- 隐层节点数目：
  - 根据具体问题进行试探选择
  - 据对问题的先验知识去精心地设计隐层节点的层数和节点数目
  - 用算法来确定隐层节点数目

多层感知器网络用于模式识别

- 输入x—— 样本特征向量（必要时归一化）
- 输出Y—— 类别编码，实现分类
- 两类问题：

  一个输出结点，0、1各代表一类
- 多类问题（1-of-C编码）：

  c类则c个输出结点，第i类则$y_i = 1$，其它$y_j = 0$，$i \neq j$
- 特征预处理

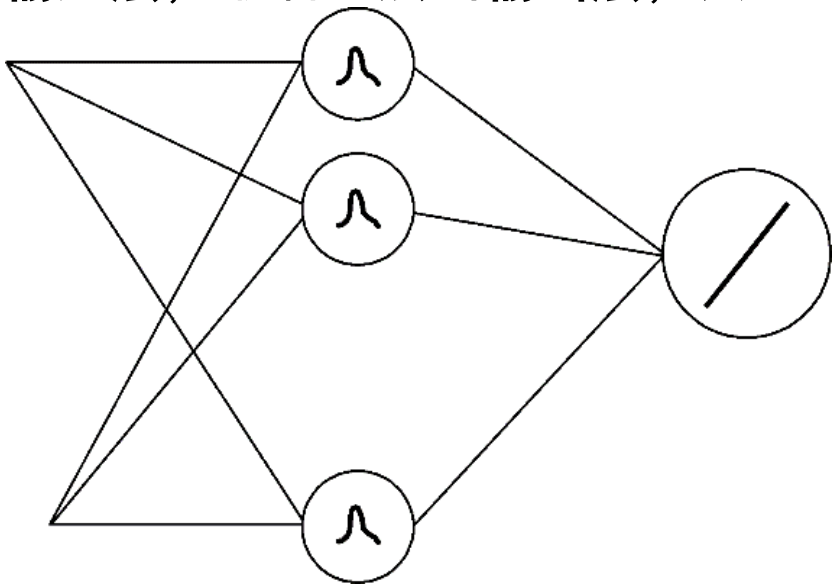
  调整特征的尺寸和均值，处于 Sigmoid函数较为灵敏的范围内

前馈神经网络与传统模式识别方法的关系

- 单层的感知器模型是采用感知准则函数的线性判别函数
- 多层则可看作它的非线性推广和发展
- 很多情况下，多层感知器输出可看作是对贝叶斯后验概率的估计

人工神经网络的知识补充

- 三种类型：
  - 前馈型网络（feedforward network）
    - 采用单向多层结构，其中每一层包含若干个神经元。在此种神经网络中，各神经元可以接收前一层神经元的信号，并产生输出到下一层。第0层叫**输入层**，最后一层叫**输出层**，其他中间层叫做隐含层（或隐藏层、隐层）
      - 多层感知器MLP
      - 径向基函数（RBF）网络
      - BP网络

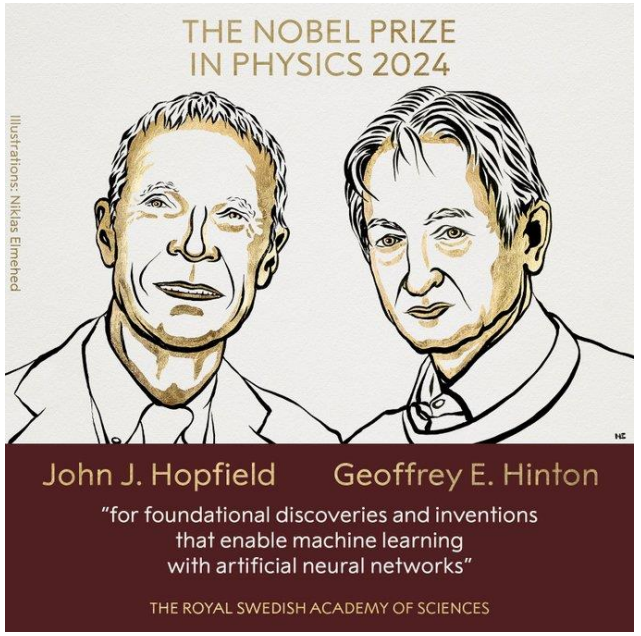


具体应用：CT扫描诊断、面部识别、垃圾邮件过滤、产品推荐系统

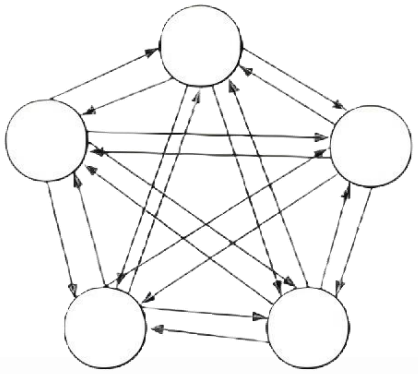
– 反馈型网络（feedback network）：

– 是一种将输出经过一步时移再接入到输入层的神经网络系统

- **Hopfield网络**：Hopfield 网络通过 **Hebbian 学习规则**计算权重矩阵来存储模式，在**状态更新公式**下演化，使得网络逐步降低能量函数，最终稳定在一个**局部最小能量**状态以实现模式记忆和检索



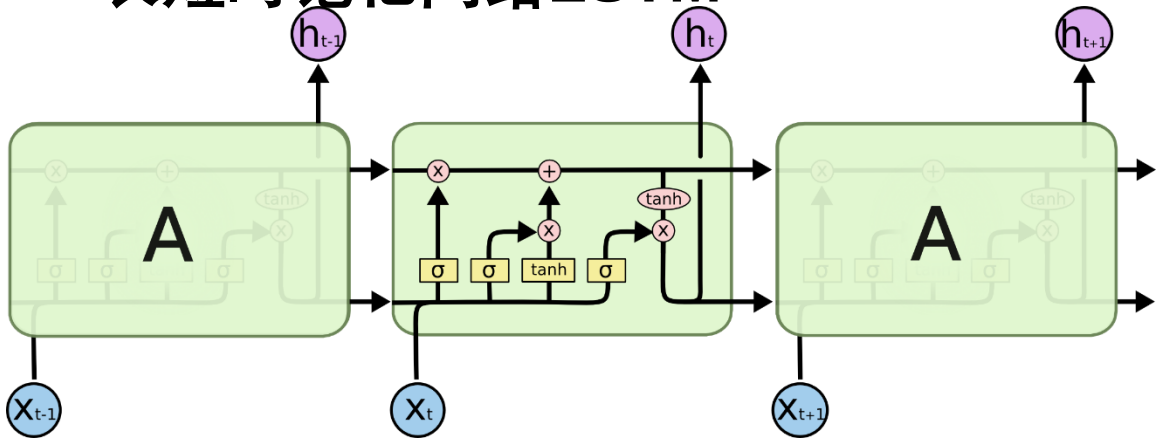
更新神经元s_i：$s_i(t+1) = sign(\sum_j w_{ij} s_j(t))$

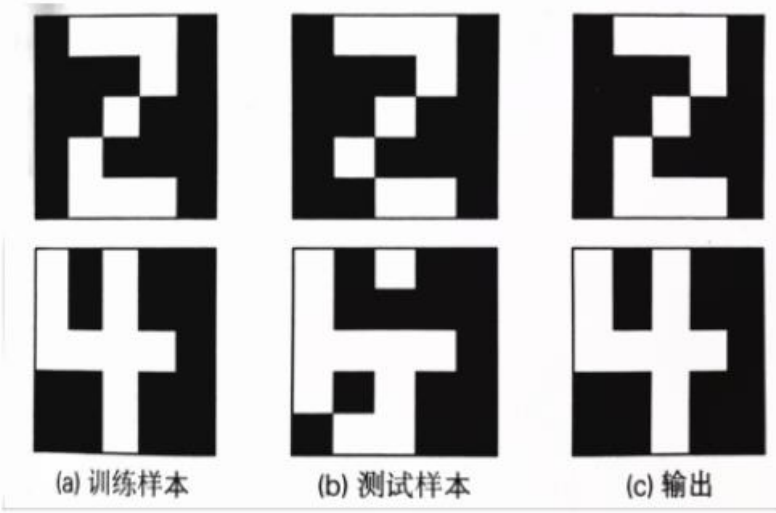


能量函数：$E = -\frac{1}{2}\sum_{ij} w_{ij} s_i s_j$

Hebbian 学习规则：$w_{ij} = \frac{1}{N}\sum_{\mu=1}^{P} \xi_i^\mu \ \xi_j^\mu$

- **长短时记忆网络LSTM**





记住训练样本的输入

↓

自联想记忆

↓

对测试样本去噪

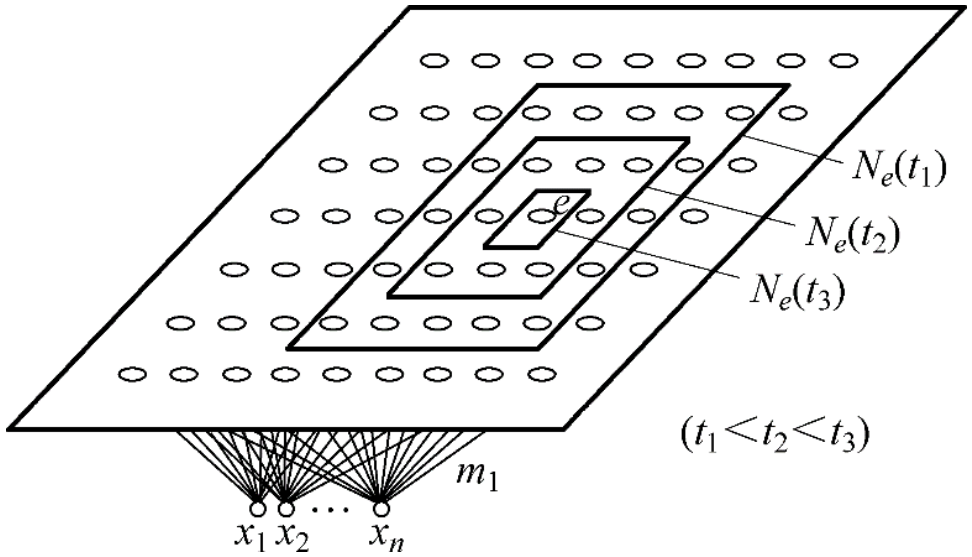
– 竞争学习网络 （competitive learning network）：

如：自组织映射网络（SOM）

SOM神经网络优势

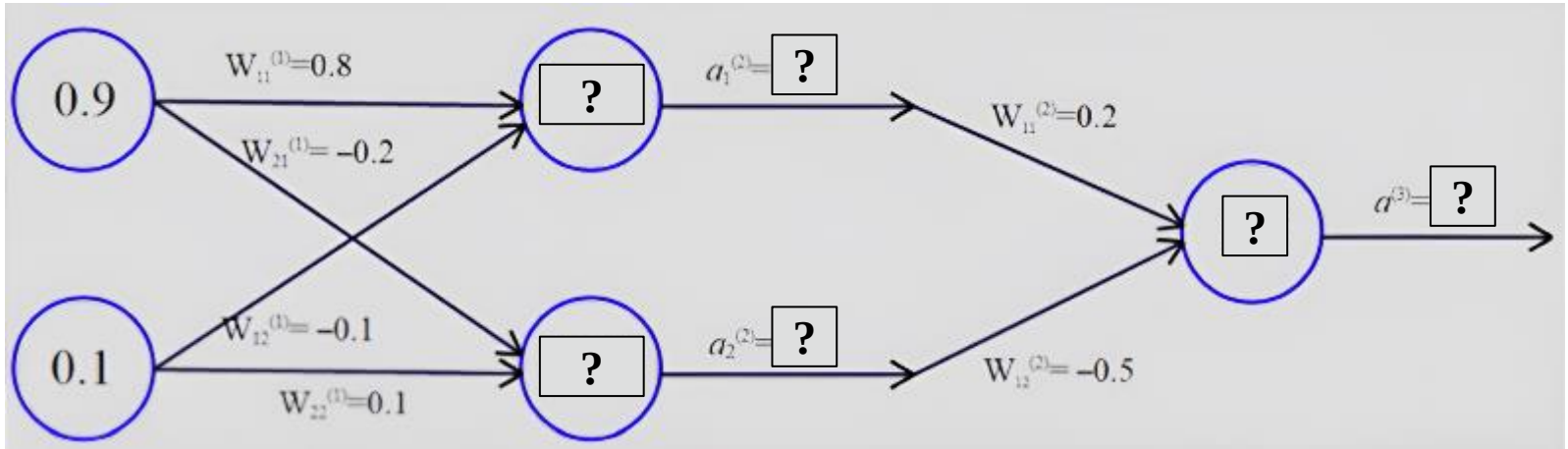
- 无监督、自组织、自学习

通过自身的训练，能自动对输入模式进行分类



区别有导师监督机制

- 课堂作业：



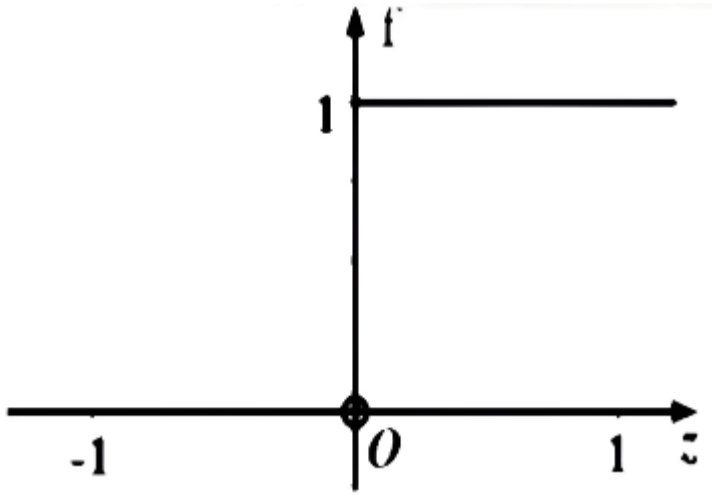
三层神经网络模型，输入层(2）、一层隐层（2）、输出层（1)， b=0

采用阶跃函数作为激活函数

$$W^{(1)} = \begin{bmatrix} 0.8 & -0.1 \\ -0.2 & 0.1 \end{bmatrix}, W^{(2)} = [0.2 - 0.5], x = \begin{bmatrix} 0.9 \\ 0.1 \end{bmatrix},$$

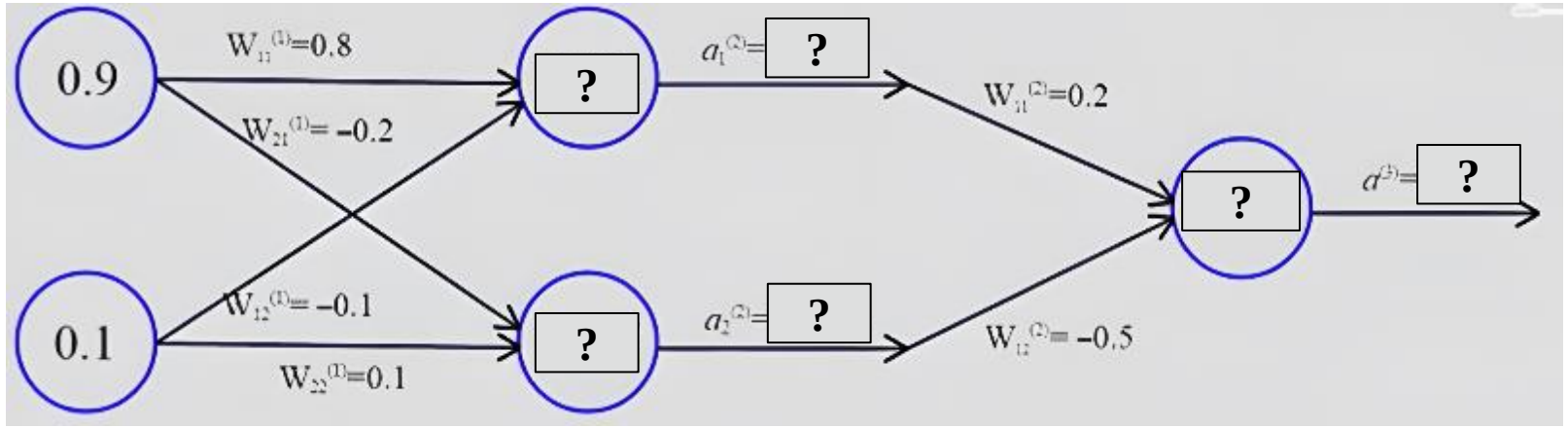
$$z^{(2)} = W^{(1)}x = \begin{bmatrix} 0.71 \\ -0.17 \end{bmatrix}, \quad a^{(2)} = \sigma(z^{(2)}) = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

$$z^{(3)} = W^{(2)}a^{(2)} = 0.2, \qquad a^{(3)} = \sigma(z^{(3)}) = 1,$$



请将上述三层神经网络模型的激活函数更换为sigmoid函数后，计算输出值

- 课堂作业：



三层神经网络模型，输入层(2）、一层隐层（2）、输出层（1)， b=0

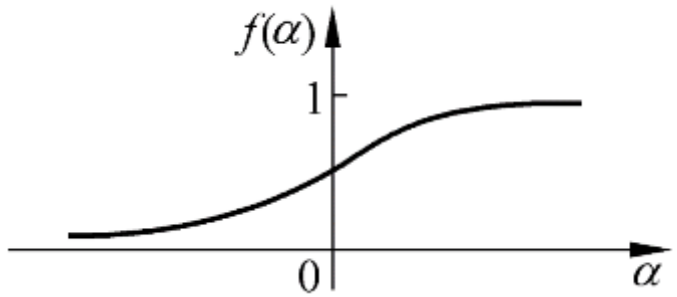
采用sigmoid函数作为激活函数

$$W^{(1)} = \begin{bmatrix} 0.8 & -0.1 \\ -0.2 & 0.1 \end{bmatrix}, \quad W^{(2)} = [0.2 - 0.5], \quad x = \begin{bmatrix} 0.9 \\ 0.1 \end{bmatrix},$$

$$z^{(2)} = W^{(1)}x = \begin{bmatrix} 0.71 \\ -0.17 \end{bmatrix}, \quad a^{(2)} = \sigma(z^{(2)}) = \begin{bmatrix} 0.6704 \\ 0.4576 \end{bmatrix},$$

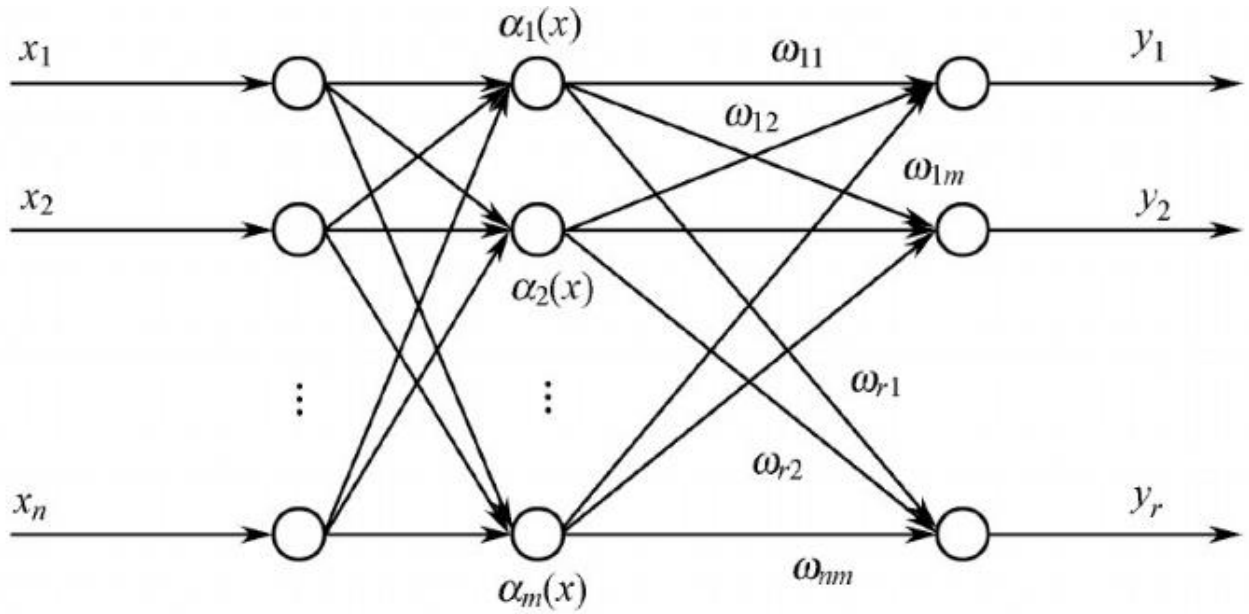
$$z^{(3)} = W^{(2)}a^{(2)} = -0.0947, \quad a^{(3)} = \sigma(z^{(3)}) = 0.4763,$$

$$f(\alpha) = \frac{1}{1 + e^{-\alpha}}$$



4.8 径向基函数网络

在模式识别等系统中,多层前馈网络是应用较为广泛的模型 。但是大部分基于反向传播的多层前馈网络的学习算法(例如BP)必须基于某种非线性优化技术的缺点 ,计算量大、学习速度慢。**径向基函数神经网络**(Radial Basis Function Neural Network)理论为多层前馈网络的学习提供了一种新颖而有效的手段 。RBF 网络不仅具有良好的推广能力, 而且计算量少 ,学习速度一般也比其它算法快得多



径向基函数的网络结构

https://blog.csdn.net/weixin_42555080

RBF的定义：

RBF是一种函数，其值仅依赖于**输入向量与特定中心点之间的距离**

- **局部响应**： RBF 激活依赖于输入与中心距离，对局部输入响应显著，适合非线性问题
- **训练速度快**： RBF 网络只需调整中心点和输出权重，训练速度较快。
- **高效的非线性映射**： RBF 隐藏层可将输入非线性映射到高维空间，使线性不可分的数据在输出层中可分

Gauss(高斯)函数：$\varphi(r) = \exp(-\frac{r^2}{2\sigma^2})$

反演S型函数：$\varphi(r) = \frac{1}{1+\exp(\frac{r^2}{\sigma^2})}$

拟多二次函数：$\varphi(r) = \frac{1}{(r^2+\sigma^2)^{\frac{1}{2}}}$

RBF神经网络与BP神经网络之间的区别：

1、局部逼近与全局逼近：

BP神经网络的隐节点采用**输入模式与权向量的内积**作为激活函数的自变量，而激活函数采用Sigmoid函数。各调参数对BP网络的输出具有同等地位的影响，因此BP神经网络是对非线性映射的**全局逼近**

RBF神经网络的隐节点采用**输入模式与中心向量的距离**（如欧式距离）作为函数的自变量，并使用径向基函数（如Gaussian函数）作为激活函数。神经元的输入离径向基函数中心越远，神经元的激活程度就越低（高斯函数）。RBF网络的输出与部分调参数有关，譬如，一个w_{ij}值只影响一个y_i的输出（参考上面第二章网络输出），RBF神经网络因此具有**“局部映射”**特性

当神经网络的一个或多个可调参数(权值和阈值)对任何一个输出都有影响，则称该神经网络为全局逼近网络

全局逼近 → 学习速度很慢，无法满足实时性要求的应用

局部逼近 → 学习速度快，可能满足有实时性要求的应用

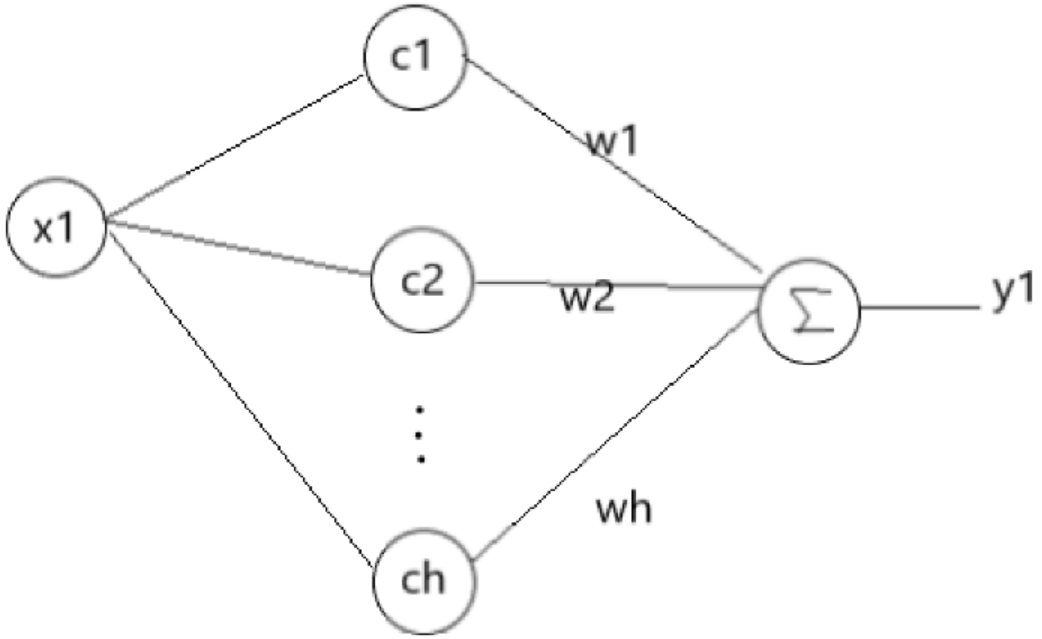
对网络输入空间的某个局部区域只有少数几个连接权影响网络的输出，则称该网络为局部逼近网络

2、中间层数的区别

BP神经网络可以有多个隐含层，但是RBF只有一个隐含层

3、训练速度的区别

使用RBF的训练速度快，一方面是因为隐含层较少，另一方面，局部逼近可以简化计算量。对于一个输入x，只有部分神经元会有响应，其他的都近似为0，对应的w就无需调参



4.9 支持向量机：非线性情形

- 上一章的最优分类超平面，即线性的支持向量机
  - 广义线性判别函数通过构造新的高维特征向量，这样可用线性判别函数解决非线性问题
- 本章讨论如何构造非线性的支持向量机

4.9.1 广义线性判别函数

- 例子：

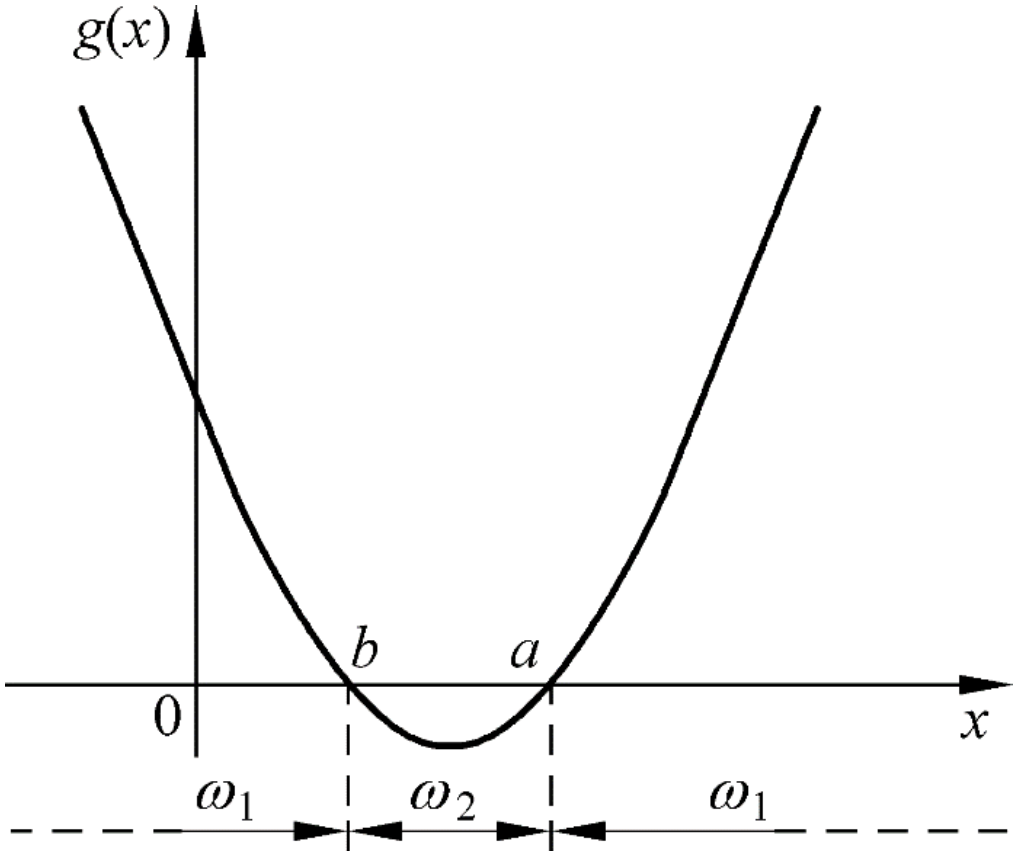
$$g(\mathrm{x}) = (x-a)(x-b)$$

- 决策规则：

$$\text{若} g(\mathrm{x}) \gtrless 0\text{，则} x \in \begin{cases} \omega_1 \\ \omega_2 \end{cases}$$

- 二次判别函数：

$$g(\mathrm{x}) = c_0 + c_1 x + c_2 x^2$$



- 转化为y的线性函数：

$$g(\mathrm{x}) = \boldsymbol{a}^{\boldsymbol{T}}\boldsymbol{y} = \sum_{i=1}^{s} a_i y_i \,,$$

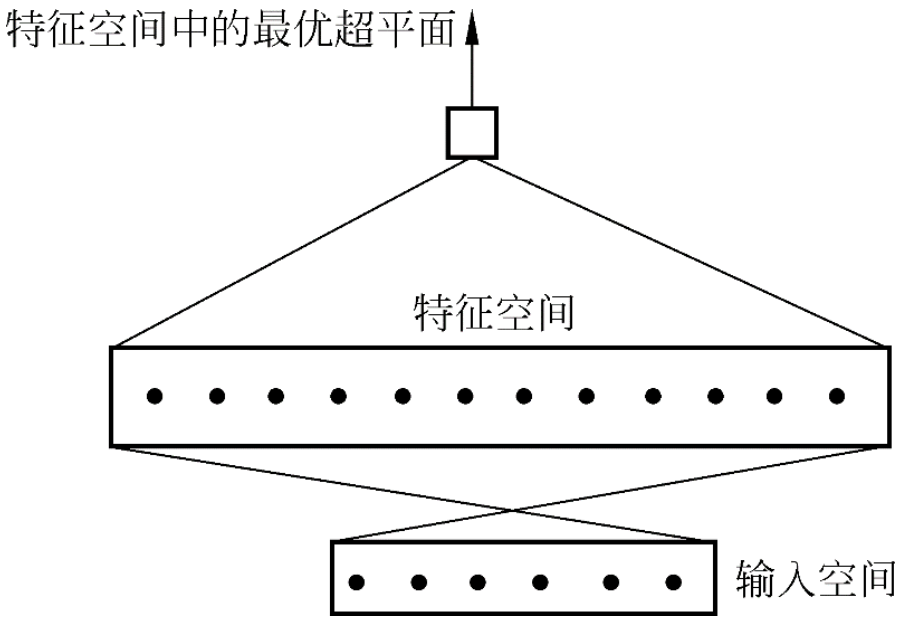
$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 1 \\ x \\ x^2 \end{bmatrix}, \boldsymbol{a} = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} c_0 \\ c_1 \\ c_2 \end{bmatrix}$$

$g(\mathrm{x}) = \boldsymbol{a}^{\boldsymbol{T}}\boldsymbol{y}$称为广义线性判别函数

- 经过变换，维数大大增加，会陷入所谓“维数灾难”（the curse of dimensionality）

4.9.2 核函数变换与支持向量机

- 支持向量机
  - 引入特征变换将非线性问题转化成线性问题



- 线性支持向量机的对偶问题：

$$\max_{\alpha} \quad Q(\boldsymbol{\alpha}) = \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{N} \alpha_i \alpha_j y_i y_j (\boldsymbol{x_i} \cdot \boldsymbol{x_j})$$

$$\text{s.t.} \quad \sum_{i=1}^{N} y_i \alpha_i = 0$$

$$0 \le \alpha_i \le C, \quad i = 1, \cdots, N$$

55

- 判别函数：

$$f(x) = sgn\{(\boldsymbol{w} \cdot \boldsymbol{x}) + b\} = sgn\left\{\sum_{i=1}^{N} \alpha_i y_i (\boldsymbol{x_i} \cdot \boldsymbol{x}) + b\right\}$$

- 支持向量满足等式：

$$y_i \left(\sum_{i=1}^{n} \alpha_i (\boldsymbol{x_i} \cdot \boldsymbol{x_j}) + b\right) - 1 = 0$$

- 特征$\boldsymbol{x}$进行非线性变换为$z = \varphi(\boldsymbol{x})$，则决策函数：

$$f(x) = sgn(\boldsymbol{w^{\varphi}} \cdot \boldsymbol{z} + b) = sgn\left(\sum_{i=1}^{n} \alpha_i y_i (\varphi(\boldsymbol{x_i}) \cdot \varphi(\boldsymbol{x})) + b\right)$$

- 优化问题变成：

$$\max_{\alpha} \quad Q(\boldsymbol{\alpha}) = \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{N} \alpha_i \alpha_j y_i y_j \left(\varphi(\boldsymbol{x_i}) \cdot \varphi(\boldsymbol{x_j})\right)$$

$$\text{s.t.} \quad \sum_{i=1}^{N} y_i \alpha_i = 0$$

$$0 \le \alpha_i \le C, \quad i = 1, \cdots, N$$

- 支持向量满足等式：

$$y_j \left(\sum_{i=1}^{n} \alpha_i y_i \left(\varphi(\boldsymbol{x_i}) \cdot \varphi(\boldsymbol{x_j})\right) + b \right) - 1 = 0$$

- 核函数

$$K(\boldsymbol{x_i}, \boldsymbol{x_j}) \overset{def}{=} \left(\varphi(\boldsymbol{x_i}) \cdot \varphi(\boldsymbol{x_j})\right)$$

- 变换空间里的支持向量机：

$$f(x) = sgn\left(\sum_{i=1}^{n} \alpha_i y_i K(\boldsymbol{x_i}, \boldsymbol{x}) + b\right)$$

- 系数α是下列优化问题的解：

$$\max_{\alpha} \quad Q(\boldsymbol{\alpha}) = \sum_{i=1}^{N} \alpha_i - \frac{1}{2}\sum_{i,j=1}^{N} \alpha_i \alpha_j y_i y_j K(\boldsymbol{x_i}, \boldsymbol{x_j})$$

$$\text{s.t.} \quad \sum_{i=1}^{N} y_i \alpha_i = 0$$

$$0 \le \alpha_i \le C, \quad i = 1, \cdots, N$$

- b通过满足下式的样本（支持向量）求得：

$$y_j\left(\sum_{i=1}^{n} \alpha_i y_i K(\boldsymbol{x_i}, \boldsymbol{x_j}) + b\right) - 1 = 0$$

- 定理（Mercer条件）
  - 对于任意的对称函数$K(\boldsymbol{x},\boldsymbol{x}')$，它是某个特征空间中的内积运算的充分必要条件是，对任意的$\varphi \neq 0$且$\int \varphi^2(\boldsymbol{x})d\boldsymbol{x} < \infty$，有

$$\iint K(\boldsymbol{x},\boldsymbol{x}')\varphi(\boldsymbol{x})\varphi(\boldsymbol{x}')d\boldsymbol{x}d\boldsymbol{x}' > 0$$

- 正定核（positive definite kernels）：
  - $K(x_i,x_j)$是定义在空间X上的对称函数，且对任意的训练数据$x_1,\cdots,x_m \in X$和任意的实系数$a_1,\cdots,a_m \in R$，都有

$$\sum_{i,j} a_i a_j K(x_i,x_j) \geq 0$$

- 肯定存在$K(\boldsymbol{x},\boldsymbol{x}') = \big(\varphi(\boldsymbol{x})\varphi(\boldsymbol{x}')\big)$的空间称为可再生核希尔伯特空间RKHS（reproducing kernel Hilbert space）

- 常用核函数形式：
  - 多项式核函数

$$K(\boldsymbol{x}, \boldsymbol{x}') = ((\boldsymbol{x} \cdot \boldsymbol{x}') + 1)^q$$

  - 径向基（RBF）核函数

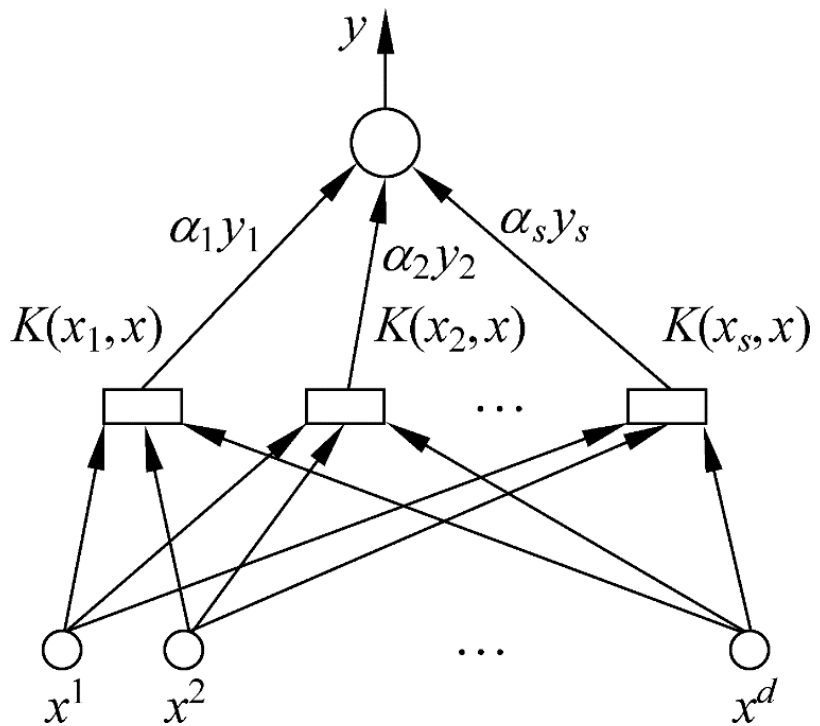
$$K(\boldsymbol{x}, \boldsymbol{x}') = exp\left(-\frac{\|\boldsymbol{x} - \boldsymbol{x}'\|^2}{\sigma^2}\right)$$

  - Sigmoid函数

$$K(\boldsymbol{x}, \boldsymbol{x}') = tanh(v(\boldsymbol{x} \cdot \boldsymbol{x}') + c)$$

（v, c需满足一定条件）

- 核函数及其参数的选择
  - 具体问题具体分析：采用启发式方法或者累试的方法
  - 参数先尝试简单的选择，再考虑非线性核
- 核函数与相似性度量
  - 不同核函数，可以看作不同相似性度量



输出(决策规则)：

$$y=\mathrm{sgn}(\sum_{i=1}^{s}\alpha_i y_i K(x_i,x)+b)$$

权值$w_i=\alpha_i y_i$

基于s个支持向量$x_1, x_2, \cdots, x_s$的非线性变换(内积)

输入向量$\boldsymbol{x}=[x^1, x^2 \cdots, x^d]$

- 维数和推广能力
  - 采用核函数作为内积，避免了高维空间的计算
  - 但核函数仍然是将特征映射到高维空间
  - 通过最大化分类间隔控制函数集的VC维

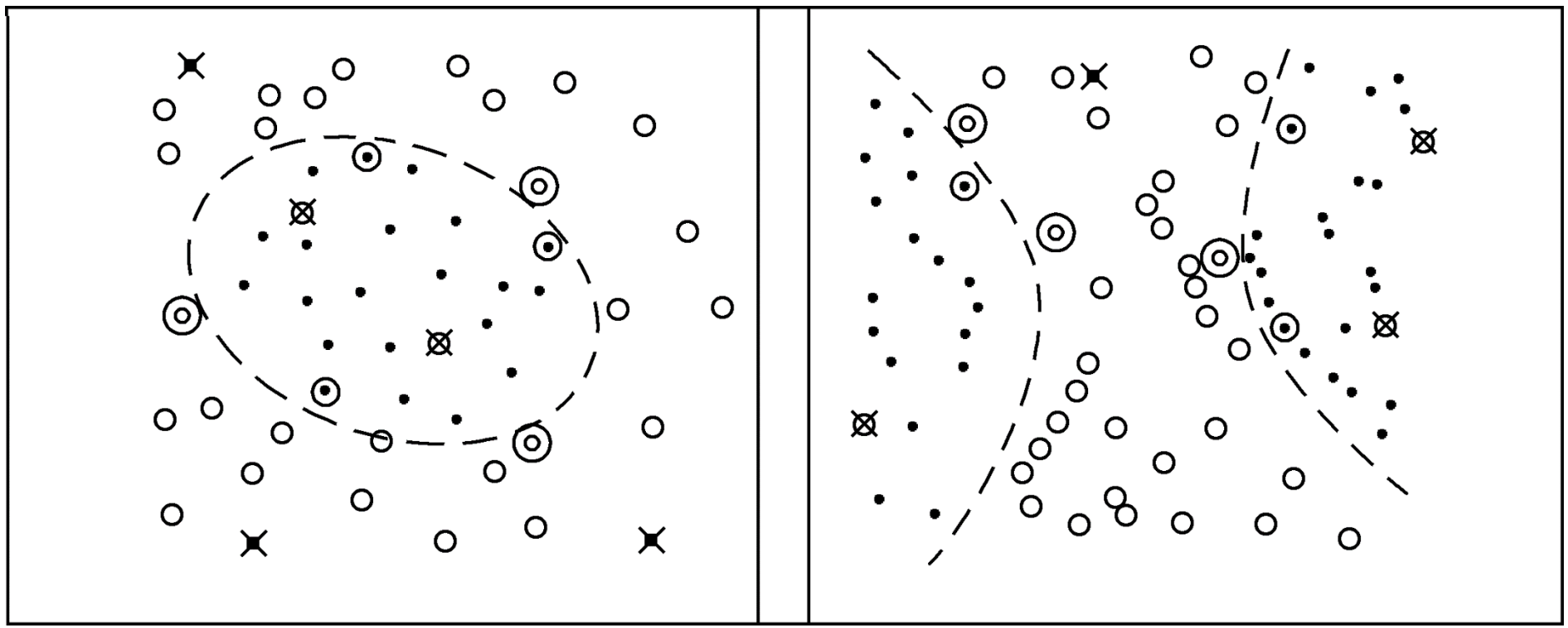
4.9.3 支持向量机早期应用举例

SVM基本思想

- 通过非线性变换将输入空间变换到一个高维空间
- 在这个新空间求解最优分类面即最大间隔分类面
- 非线性变换通过适当的内积核函数实现
- 求解一个有约束的二次优化问题，有唯一最优解，这与多层感知器网络相比是一个优势。
- 问题的计算复杂度由样本数目决定，不取决与样本变换特征维数与所采用的核函数

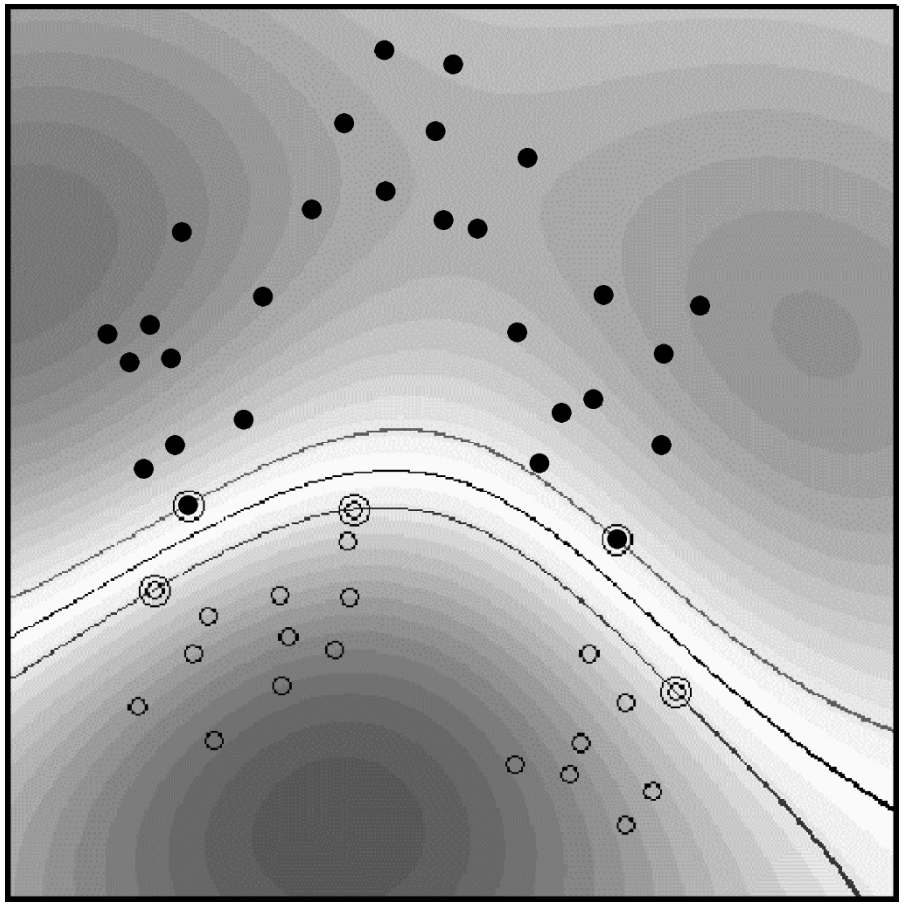
应用举例

（1）对两组数据用二阶多项式核函数



应用举例

（2）用高斯径向基核函数径向基核函数的例子

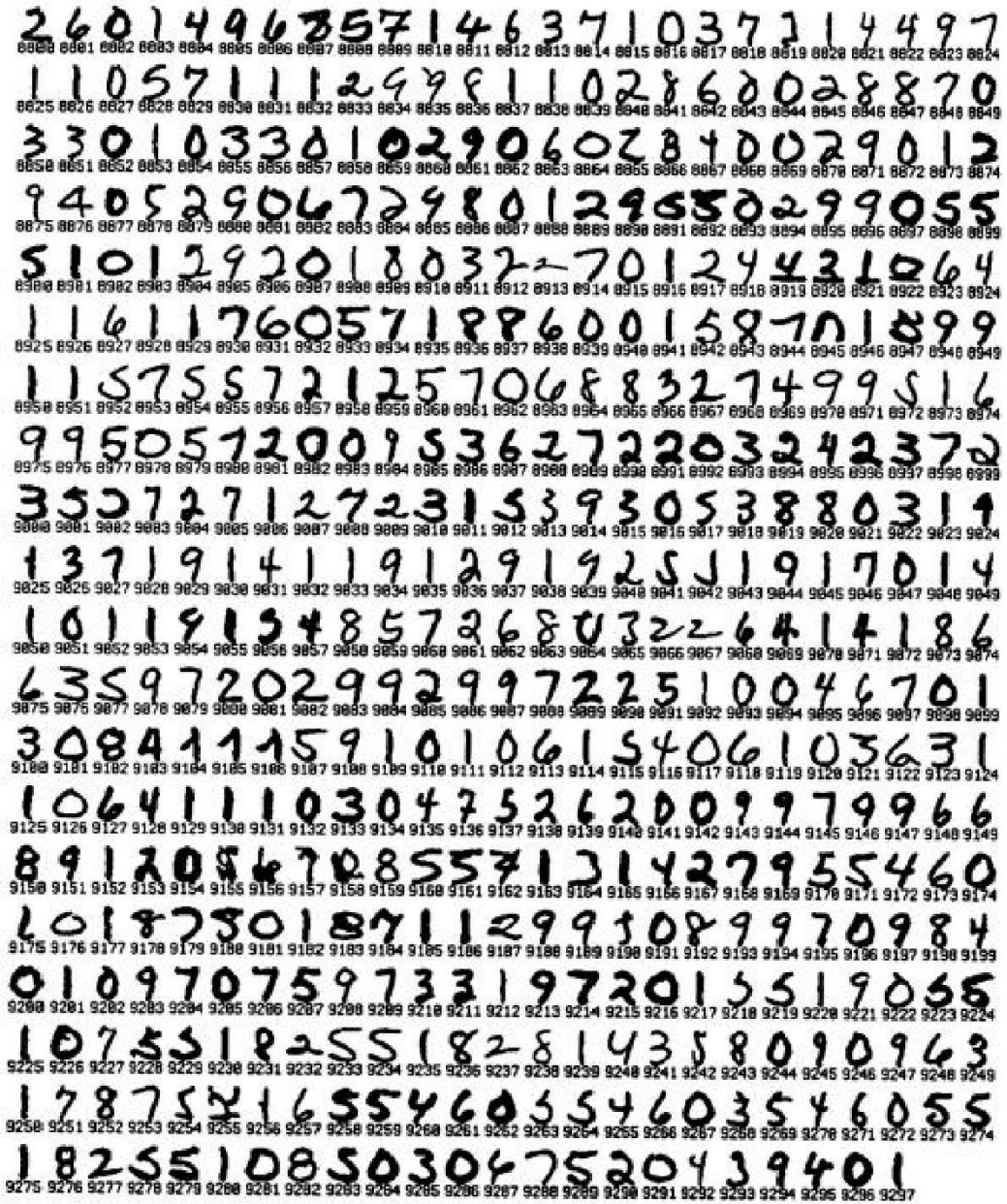


应用举例

（3）USPS手写数字样本举例：

- 传统方法错误率：

方法	测试错误率
人工分类	2.5%
决策树方法	16.2%
两层神经网络	5.9%
五层神经网络	5.1%



- 支持向量机方法：

$$\text{多项式核函数：} K(\boldsymbol{x}, \boldsymbol{x}') = \left(\frac{1}{256}(\boldsymbol{x} \cdot \boldsymbol{x}')\right)^q$$

$$\text{径向基核函数：} K(\boldsymbol{x}, \boldsymbol{x}') = exp\left(-\frac{\|\boldsymbol{x} - \boldsymbol{x}'\|^2}{256\sigma^2}\right)$$

$$Sigmoid\text{核函数：} K(\boldsymbol{x}, \boldsymbol{x}') = tanh\left(\frac{b(\boldsymbol{x} \cdot \boldsymbol{x}')}{256} - c\right)$$

方法	测试错误率
多项式内积	4.0%
径向基函数内积	4.1%
Sigmoid函数	4.2%

- 这是支持向量机的第一次实际应用。目前已有两个标准数字数据库USPS和NIST，作为测试多种分类器优劣的标准
- USPS数据库包括7291个训练样本和2007测试样本，每个样本的输入为手写阿拉伯数字的图像，16×16个像素，每个像素灰度为0到256。即每个样本的输入可用16×16＝256维的向量表示，其中每个分量为灰度值
- NIST数据库包括60000个训练样本和10000个测试样本，每个样本的输入为20×20个像素。同样，每个样本的输入可用400维的向量表示，每个分量也是0到256的灰度值

应用举例

（4）文本分类－根据文本内容自动归类

邮件过滤、网页搜索及办公自动化中应用

- 利用支持向量机对12902个文本进行研究，其中9603个为训练样本，3299为测试样本。每个文本约包含200个单词，分属118类。
- 预处理去掉无关单词，留下9947个词根，按词根顺序组成“字典”，每个文本根据“字典”表示为9947维向量

$$\boldsymbol{X}=[(\boldsymbol{x})_1,\cdots,(\boldsymbol{x})_{9947}]^{\boldsymbol{T}}$$

$$(\boldsymbol{x})_i=\frac{\boldsymbol{t}_i\log_2(\boldsymbol{r}_i)}{\boldsymbol{k}}\qquad \boldsymbol{i}=1,2,\cdots,9947$$

$\boldsymbol{t}_i$ – 文本中第$\boldsymbol{i}$个词根出现的次数；

$\boldsymbol{r}_i$ – 所有文本的个数与包含第$\boldsymbol{i}$个词根文本数之比；

$\boldsymbol{k}$ – 调整$\boldsymbol{x}$使其$\|\boldsymbol{x}\|=1$的尺寸。

核函数为线性核函数

$$\boldsymbol{K}(\boldsymbol{x},\boldsymbol{x}')=(\boldsymbol{x}\cdot\boldsymbol{x}')$$

4.9.4 支持向量机的实现算法

- SVM^{light} ：

  http://svmlight.joachims.org/

- SVMTorch：

  http://bengio.abracadoudou.com/projects/SVMTorch.html

- LibSVM：

  https://www.csie.ntu.edu.tw/~cjlin/libsvm/

- 更多软件：

  http://www.kernel-machines.org/software

4.9.5 多类支持向量机

- 多类支持向量机（multicategory SVM）：
  - 直接设计多类SVM分类器的方法
  - 例如用正则化（regularization）的框架重新表述来实现

4.9.6 用于函数拟合的支持向量机

- 也称作支持向量回归（support vector regression，SVR）
  - 线性回归函数

$$f(\boldsymbol{x}) = \boldsymbol{\omega} \cdot \boldsymbol{x} + b$$

  - 拟合训练数据$\{(\boldsymbol{x_i}, y_i), i = 1, \cdots, n\}$，在精度$\varepsilon$范围内：

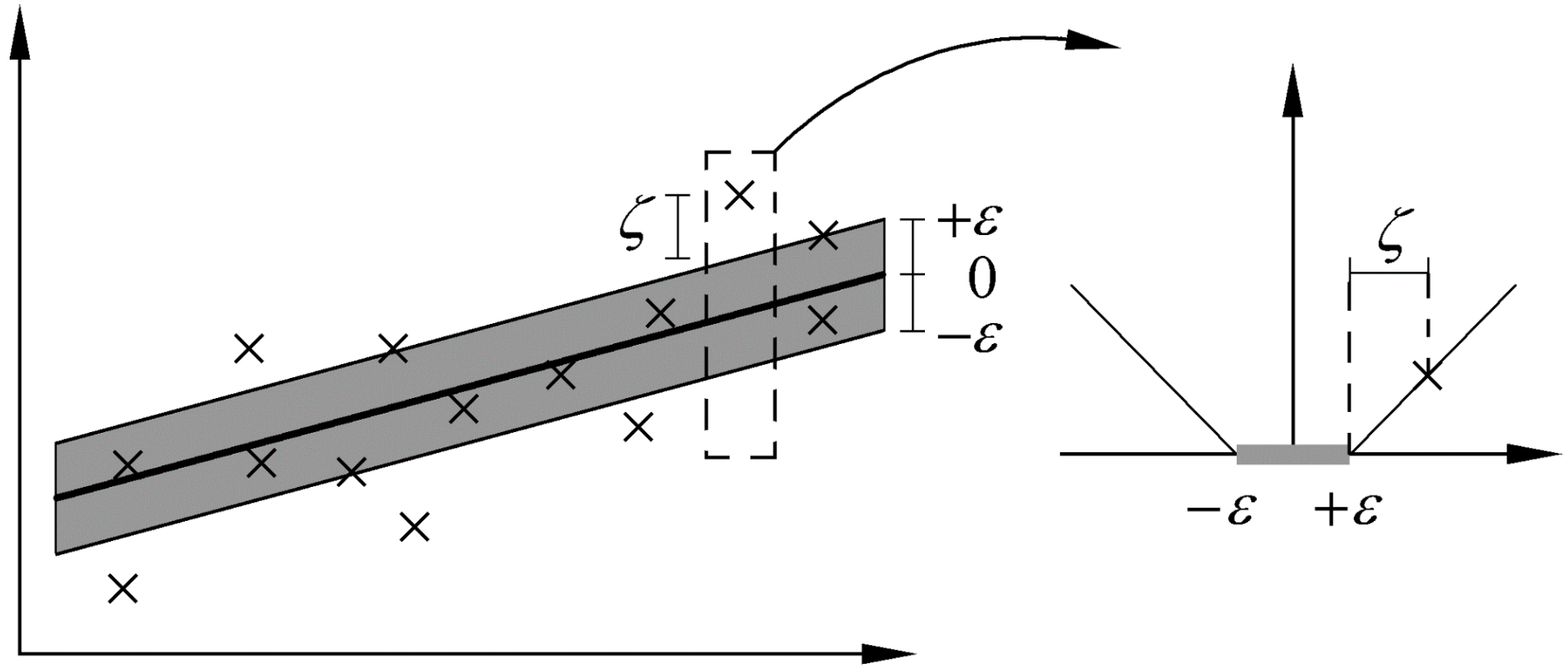
$$\begin{cases} y_i - \boldsymbol{\omega} \cdot \boldsymbol{x_i} - b \leq \varepsilon + \xi_i^*, \\ \boldsymbol{\omega} \cdot \boldsymbol{x_i} + b - y_i \geq \varepsilon + \xi_i, \end{cases} \quad i = 1, \cdots, n$$

  - 目标函数：

$$\min_{\omega, b} \frac{1}{2} \|\boldsymbol{\omega}\|^2 + C \sum_{i=1}^{n} (\xi_i + \xi_i^*)$$

$-\varepsilon-$不敏感损失函数：

$$|y-f(\boldsymbol{x},a)|_\varepsilon=\begin{cases}0, & \text{若}|y-f(\boldsymbol{x},a)|\le\varepsilon\\ |y-f(\boldsymbol{x},a)|-\varepsilon, & \text{其他}\end{cases}$$



– 对偶问题：

$$\max_{\boldsymbol{\alpha},\boldsymbol{\alpha}^*} W(\boldsymbol{\alpha},\boldsymbol{\alpha}^*) = -\varepsilon\sum_{i=1}^{l}(\alpha_i^* + \alpha_i) + \sum_{i=1}^{l} y_i(\alpha_i^* - \alpha_i)$$

$$-\frac{1}{2}\sum_{i,j=1}^{l}(\alpha_i^* - \alpha_i)\left(\alpha_j^* - \alpha_j\right)\left(\boldsymbol{x_i}\cdot\boldsymbol{x_j}\right)$$

$$s.t. \quad \sum_{i=1}^{l}\alpha_i^* = \sum_{i=1}^{l}\alpha_i$$

$$0 \le \alpha_i^* \le C, \quad i = 1,\cdots,l$$

$$0 \le \alpha_i \le C, \quad i = 1,\cdots,l$$

– 权值和系数的关系：

$$\boldsymbol{\omega} = \sum_{i=1}^{l}\left(\alpha_j^* - \alpha_j\right)\boldsymbol{x_i}$$

– 回归函数：

$$f(\boldsymbol{x}) = \boldsymbol{\omega} \cdot \boldsymbol{x} + b = \sum_{i=1}^{n} \left(\alpha_j^* - \alpha_j\right)\left(\boldsymbol{x_i} \cdot \boldsymbol{x_j}\right) + b^*$$

– 核函数变换得到拟合函数：

$$f(\boldsymbol{x}) = \sum_{i=1}^{l} \beta_i K\left(\boldsymbol{x_i} \cdot \boldsymbol{x_j}\right) + b$$

– 其中$\beta_i \overset{def}{=} \alpha_i^* - \alpha_i$是以下优化问题的解：

$$\max_{\boldsymbol{\alpha},\boldsymbol{\alpha}^*} W(\boldsymbol{\alpha}, \boldsymbol{\alpha}^*) = -\varepsilon \sum_{i=1}^{l} (\alpha_i^* + \alpha_i) + \sum_{i=1}^{l} y_i(\alpha_i^* - \alpha_i) - \frac{1}{2} \sum_{i,j=1}^{l} (\alpha_i^* - \alpha_i)\left(\alpha_j^* - \alpha_j\right) K\left(\boldsymbol{x_i} \cdot \boldsymbol{x_j}\right)$$

$$s.t. \quad \sum_{i=1}^{l} \alpha_i^* = \sum_{i=1}^{l} \alpha_i \,, \;\; 0 \le \alpha_i^* \le C, \;\; i = 1, \cdots, l, \;\; 0 \le \alpha_i \le C, \;\; i = 1, \cdots, l$$

4.9.7 核函数机器
大间隔机器与核函数机器

- 非线性变换存在的问题：
  - 计算层面：维数过高
  - 概念层面：面对新样本推广能力差
- 支持向量机核心思想：
  - 最大化分类间隔保证推广能力
  - 核函数实现特征的非线性变换
- 大间隔方法（Large-margin methods）
- 核函数方法（Kernel methods）

核Fisher判别

- 核Fisher判别（Kernel Fisher's Discriminant）
  - Fisher线性判别

$$\max_{w} J(\boldsymbol{w}) = \frac{\boldsymbol{w}^T \boldsymbol{S}_B \boldsymbol{w}}{\boldsymbol{w}^T \boldsymbol{S}_w \boldsymbol{w}}$$

$$\boldsymbol{S}_B = (\boldsymbol{m}_1 - \boldsymbol{m}_2)(\boldsymbol{m}_1 - \boldsymbol{m}_2)^T$$

$$\boldsymbol{S}_w = \sum_{q=1,2} \sum_{x_i \epsilon \omega_q} (\boldsymbol{x}_i - \boldsymbol{m}_q)(\boldsymbol{x}_i - \boldsymbol{m}_q)^T$$

  - 其解为：

$$\boldsymbol{w} = {\boldsymbol{S}_w}^{-1}(\boldsymbol{m}_1 - \boldsymbol{m}_2)$$

– 样本x非线性变换：

$$J(\boldsymbol{w}) = \frac{\boldsymbol{w}^T \boldsymbol{S}_B^{\Phi} \boldsymbol{w}}{\boldsymbol{w}^T \boldsymbol{S}_W^{\Phi} \boldsymbol{w}}$$

– 其中：

$$\boldsymbol{S}_B^{\Phi} = (\boldsymbol{m}_1^{\Phi} - \boldsymbol{m}_2^{\Phi})(\boldsymbol{m}_1^{\Phi} - \boldsymbol{m}_2^{\Phi})^T$$

$$\boldsymbol{S}_W^{\Phi} = \sum_{i=1,2} \sum_{x \epsilon \omega_i} (\boldsymbol{\Phi}(\boldsymbol{x}) - \boldsymbol{m}_i^{\Phi})(\boldsymbol{\Phi}(\boldsymbol{x}) - \boldsymbol{m}_i^{\Phi})^T$$

$$\boldsymbol{m}_i^{\Phi} = \frac{1}{l_i} \sum_{j=1}^{l_i} \boldsymbol{\Phi}(x_j^i)$$

– 解$\omega \in F$满足：

$$\omega = \sum_{j=1}^{l} \alpha_j \boldsymbol{\Phi}(x_j)$$

– 推出：

$$\boldsymbol{w}^T \boldsymbol{m}_i^{\Phi} = \frac{1}{l_i} \sum_{j=1}^{l} \sum_{k=1}^{l_i} \alpha_j k\left(\boldsymbol{x_j}, \boldsymbol{x_k^i}\right) = \boldsymbol{\alpha}^T \boldsymbol{M}_i$$

– 记：

$$\boldsymbol{M} = (\boldsymbol{M}_1 - \boldsymbol{M}_2)(\boldsymbol{M}_1 - \boldsymbol{M}_2)^{\boldsymbol{T}}$$

– 类间和类内离散度矩阵：

$$\boldsymbol{w}^T \boldsymbol{S}_B^{\Phi} \boldsymbol{w} = \boldsymbol{\alpha}^T \boldsymbol{M} \boldsymbol{\alpha}, \boldsymbol{w}^T \boldsymbol{S}_W^{\Phi} \boldsymbol{w} = \boldsymbol{\alpha}^T \boldsymbol{N} \boldsymbol{\alpha}$$

$$\boldsymbol{N} = \sum_{j=1,2} \boldsymbol{K}_j \left(\boldsymbol{I} - \boldsymbol{1}_{l_j}\right) \boldsymbol{K}_j^{\boldsymbol{T}}$$

– 目标函数成为：

$$J(\boldsymbol{\alpha}) = \frac{\boldsymbol{\alpha}^T \boldsymbol{M} \boldsymbol{\alpha}}{\boldsymbol{\alpha}^T \boldsymbol{N} \boldsymbol{\alpha}}$$

– 最大化上式的解是$\boldsymbol{N}^{-1}\boldsymbol{M}$ 的最大本征值的本征向量

– 最优解的方向

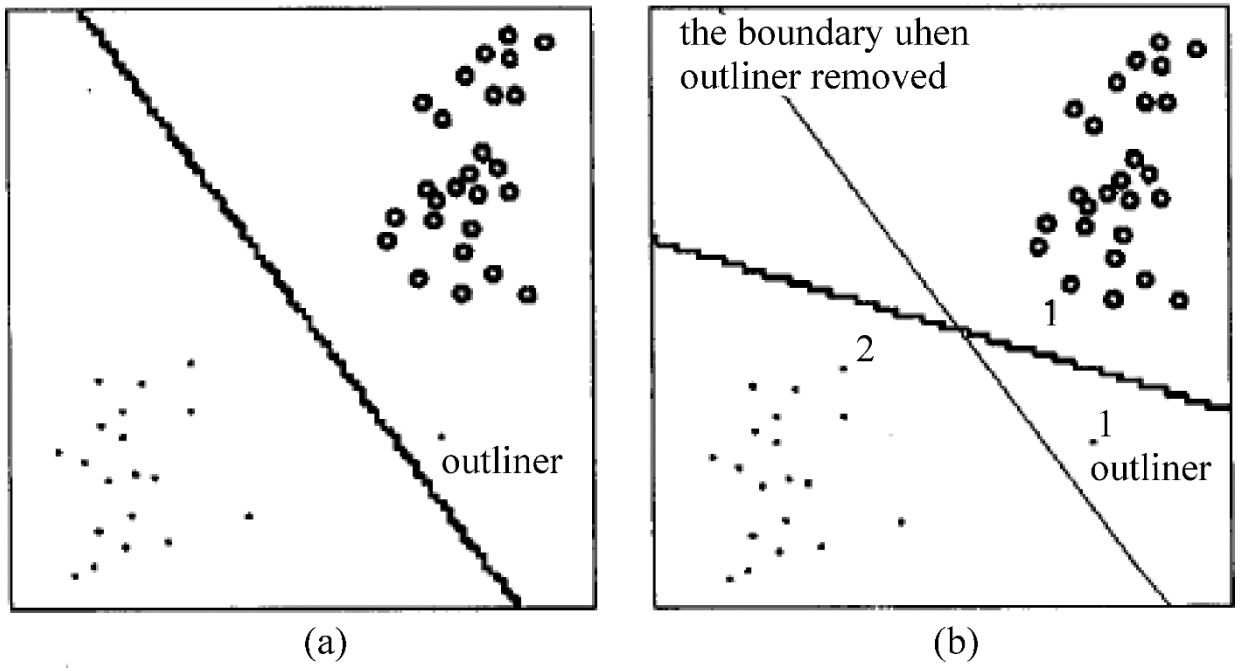
$$\boldsymbol{\alpha} \propto \boldsymbol{N}^{-1}(\boldsymbol{M}_1 - \boldsymbol{M}_2)$$

– 原空间到Fisher判别的投影：

$$< \boldsymbol{w}, \boldsymbol{\Phi}(\boldsymbol{x}) >= \sum_{i=1}^{l} \boldsymbol{\alpha}_i k(\boldsymbol{x}_i, \boldsymbol{x})$$

中心支持向量机

- 支持向量机：
  - 对样本噪声和偏离数据分布的野值非常敏感



  - 样本数非常少时，不确定性很大

- 中心支持向量机（Central SVM）：
  - 用中心间隔代替边界间隔
  - 综合基于均值和基于边界样本方法的优势
  - 极少或含野值样本下能够得到更可靠的分类器

- 线性判别函数 $y = \boldsymbol{\omega} \cdot \boldsymbol{x} + b$，　$y_i \in \{1, -1\}$，　$i = 1, \cdots, n$

（1）全部线性可分：

$$y_i(\boldsymbol{\omega} \cdot \boldsymbol{x}_i + b) > 0, i = 1, \cdots, n$$

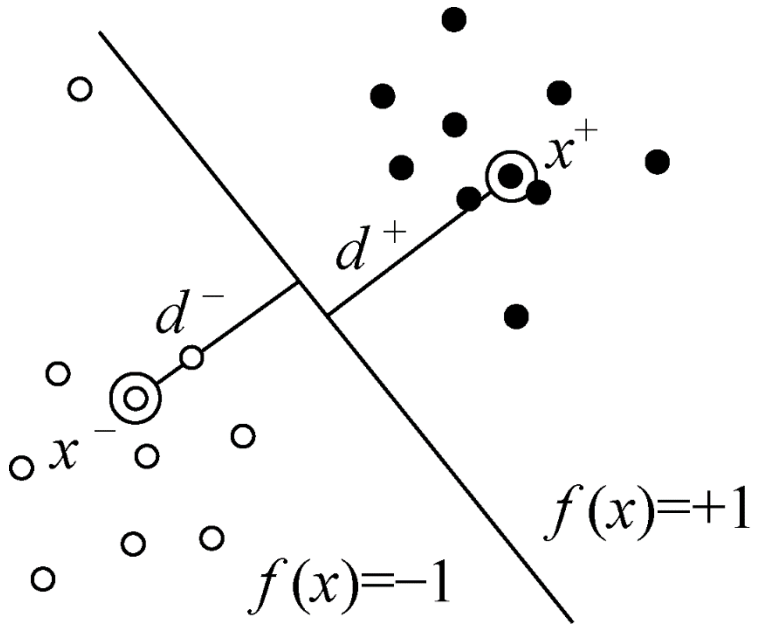
  - 引入小常数$\varepsilon > 0$:

$$y_i(\boldsymbol{\omega} \cdot \boldsymbol{x}_i + b) \geq \varepsilon > 0, i = 1, \cdots, n$$

– 记两类中心为$\boldsymbol{x}^+$和$\boldsymbol{x}^-$，到分类超平面的距离：

$$d^+ = \frac{|\boldsymbol{\omega} \cdot \boldsymbol{x}^+ + b|}{\|\boldsymbol{\omega}\|} = \frac{y^+(\boldsymbol{\omega} \cdot \boldsymbol{x}^+ + b)}{\|\boldsymbol{\omega}\|}$$

$$d^- = \frac{|\boldsymbol{\omega} \cdot \boldsymbol{x}^- + b|}{\|\boldsymbol{\omega}\|} = \frac{y^-(\boldsymbol{\omega} \cdot \boldsymbol{x}^- + b)}{\|\boldsymbol{\omega}\|}$$



– 中心分离间隔：

$$d = d^+ + d^- = \frac{\sum_{i=1}^{n} l_i y_i(\boldsymbol{\omega} \cdot \boldsymbol{x}_i + b)}{\|\boldsymbol{\omega}\|}$$

– 引入约束条件：

$$\sum_{i=1}^{n} l_i y_i(\boldsymbol{\omega} \cdot \boldsymbol{x}_i) = 1$$

– 优化问题：

$$min \frac{1}{2} \|\boldsymbol{\omega}\|^2$$

（2）线性不可分，引入$\xi_i > 0$：

$$y_i(\boldsymbol{\omega} \cdot \boldsymbol{x}_i + b) + \xi_i \geq \varepsilon > 0, i = 1, \cdots, n$$

– 目标函数修正：

$$min \frac{1}{2} \|\boldsymbol{\omega}\|^2 + C(\sum_{i=1}^{n} \xi_i)$$

– 对偶问题：

$$max\, Q(\alpha+\beta)=\sum_{i=1}^{n}\varepsilon\alpha_i+\beta-\frac{1}{2}\sum_{i,j=1}^{n}(\alpha_i+\beta l_i)(\alpha_j+\beta l_j)y_i\, y_j(\boldsymbol{x}_i\cdot\boldsymbol{x}_j)$$

– 约束条件：

$$\sum_{i=1}^{n}y_i\alpha_i=0, 0\le\alpha_i\le C, i=1,\cdots n, \beta\ge 0$$

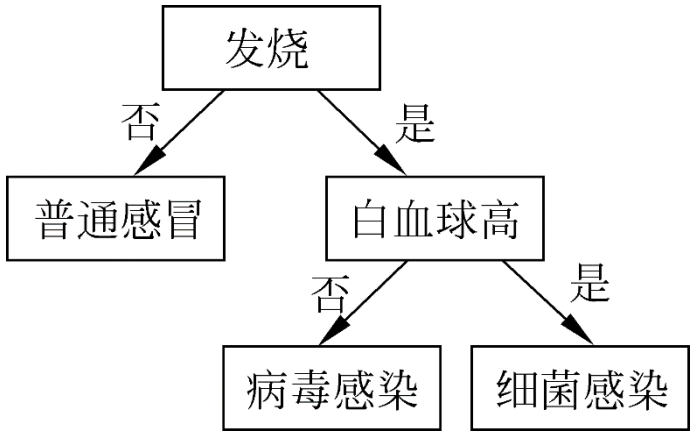
– 最优解满足：

$$\boldsymbol{\omega}^*=\sum_{i=1}^{n}(\alpha_i{}^*+\beta^* l_i)\, y_i\boldsymbol{x}_i=\sum_{i=1}^{n}\alpha_i{}^* y_i\boldsymbol{x}_i+\beta^*(x^+-x^-)$$

$$\boldsymbol{\omega}^{CSVM}=(1-\lambda)\boldsymbol{\omega}^{SVM}+\lambda(x^+-x^-)$$

4.10 决策树

- 一个简化的树状决策过程例子：



- 汽车推销员对潜在客户的分析示例（如右表）：

编号	年龄	性别	月收入	是否购买
1	21	男	4000	否
2	33	女	5000	否
3	30	女	3800	否
4	38	女	2000	否
5	25	男	7000	否
6	32	女	2500	否
7	20	女	2000	否
8	26	女	9000	是
9	32	男	5000	是
10	24	男	7000	否
11	40	女	4800	否
12	28	男	2800	否
13	35	女	4500	否
14	33	男	2800	是
15	37	男	4000	是
16	31	女	2500	否

- 初步整理后的顾客数据：
- 决策树：
  - 节点：一个特征和相应的决策规则
  - 根：最顶端的节点
  - 叶节点：只包含单纯一类的样本，不需要继续划分

顾客编号	年龄	性别	月收入	是否购买
1	<30	男	中	否
2	≥30	女	中	否
3	≥30	女	中	否
4	≥30	女	低	否
5	<30	男	高	否
6	≥30	女	低	否
7	<30	女	低	否
8	<30	女	高	是
9	≥30	男	中	是
10	<30	男	高	否
11	≥30	女	中	否
12	<30	男	低	否
13	≥30	女	中	否
14	≥30	男	低	是
15	≥30	男	中	是
16	≥30	女	低	否

ID3（interactive dichotomizer-3）方法

- 香农熵（Shannon Entropy）：

$$I = -(P_1 \log_2 P_1 + P_2 \log_2 P_2 + \cdots + P_k \log_2 P_k) = -\sum_{i=1}^{k} P_i \log_2 P_i$$

- 汽车顾客的例子里，在不考虑任何特征时：

$$I(16,4) = -\left(\frac{4}{16}\log_2\frac{4}{16} + \frac{12}{16}\log_2\frac{12}{16}\right) = 0.8113$$

- 如采用年龄作为根节点，则把所有样本分为两组：

$$I_{age} = \frac{6}{16}I(6,1) + \frac{10}{16}I(10,3) = 0.7946$$

- 不纯度减少量（信息增益Information Gain）：

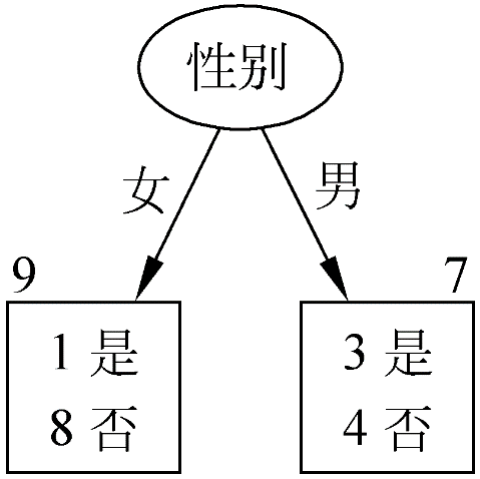
$$\Delta I_{age}(16) = I(16,4) - I_{age} = 0.0167$$

- 一般地，$\Delta I(N) = I(N) - \left(P_1 I(N_1) + P_2 I(N_2) + \cdots + P_m I(N_m)\right), P_m = \frac{N_m}{N}$
- 上例中：

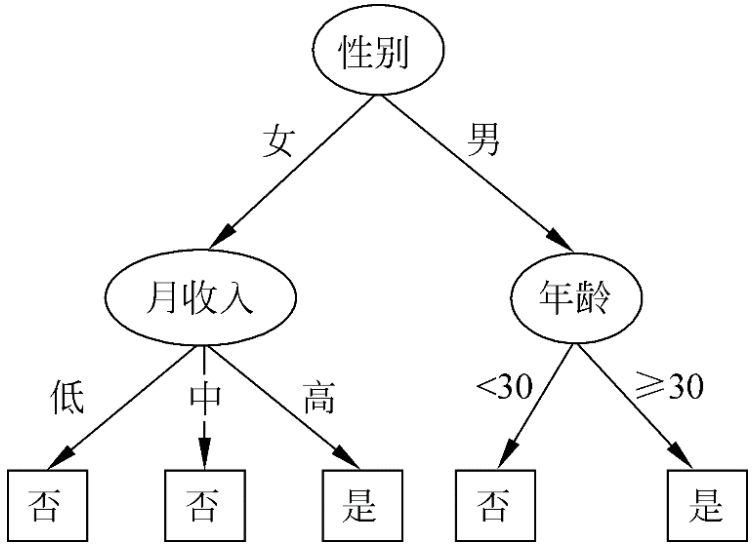
$$\Delta I_{gender}(16) = I(16,4) - I_{gender} = 0.0972$$
$$\Delta I_{income}(16) = I(16,4) - I_{income} = 0.0177$$

选取性别作第一个特征能够带来不纯度最大的减小

于是得到：

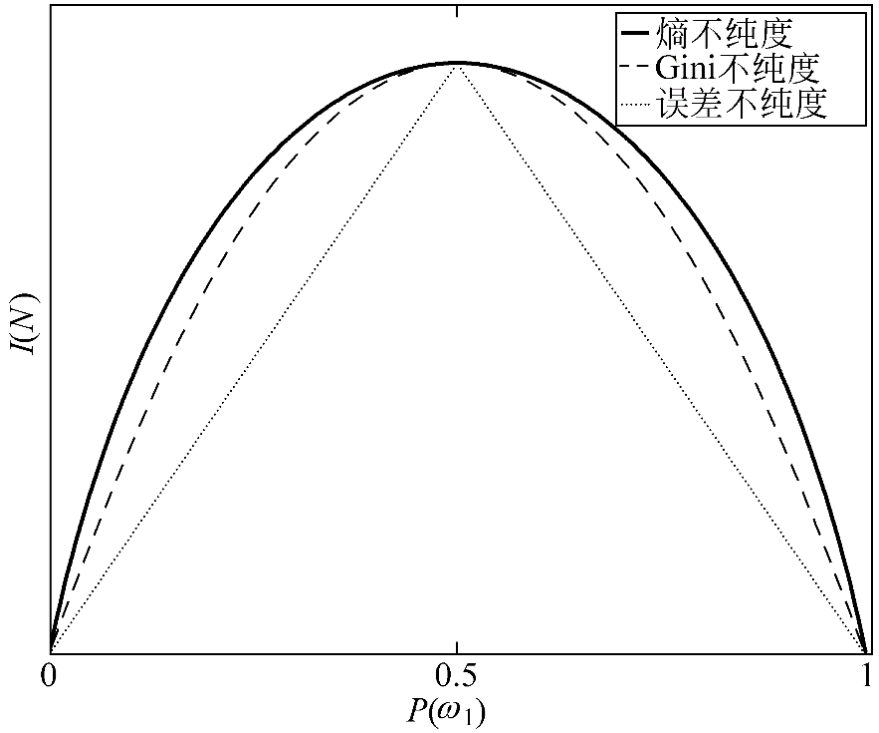


同理可得：



其他决策树算法

- 其他不纯度度量：
  - Gini不纯度（方差不纯度）：$I(N) = \sum_{m \neq n} P(\omega_m)\, P(\omega_n) = 1 - \sum_{j=1}^{k} P^2(\omega_j)$
  - 误差不纯度：$I(N) = 1 - \max_j P(\omega_j)$
- C4.5算法：
  - 以信息增益率代替信息增益：$\Delta I_R(N) = \frac{\Delta I(N)}{I(N)}$
  - 能够处理连续数值特征
- CART算法：
  - 每个节点上都采用二分法，最后构成二叉树

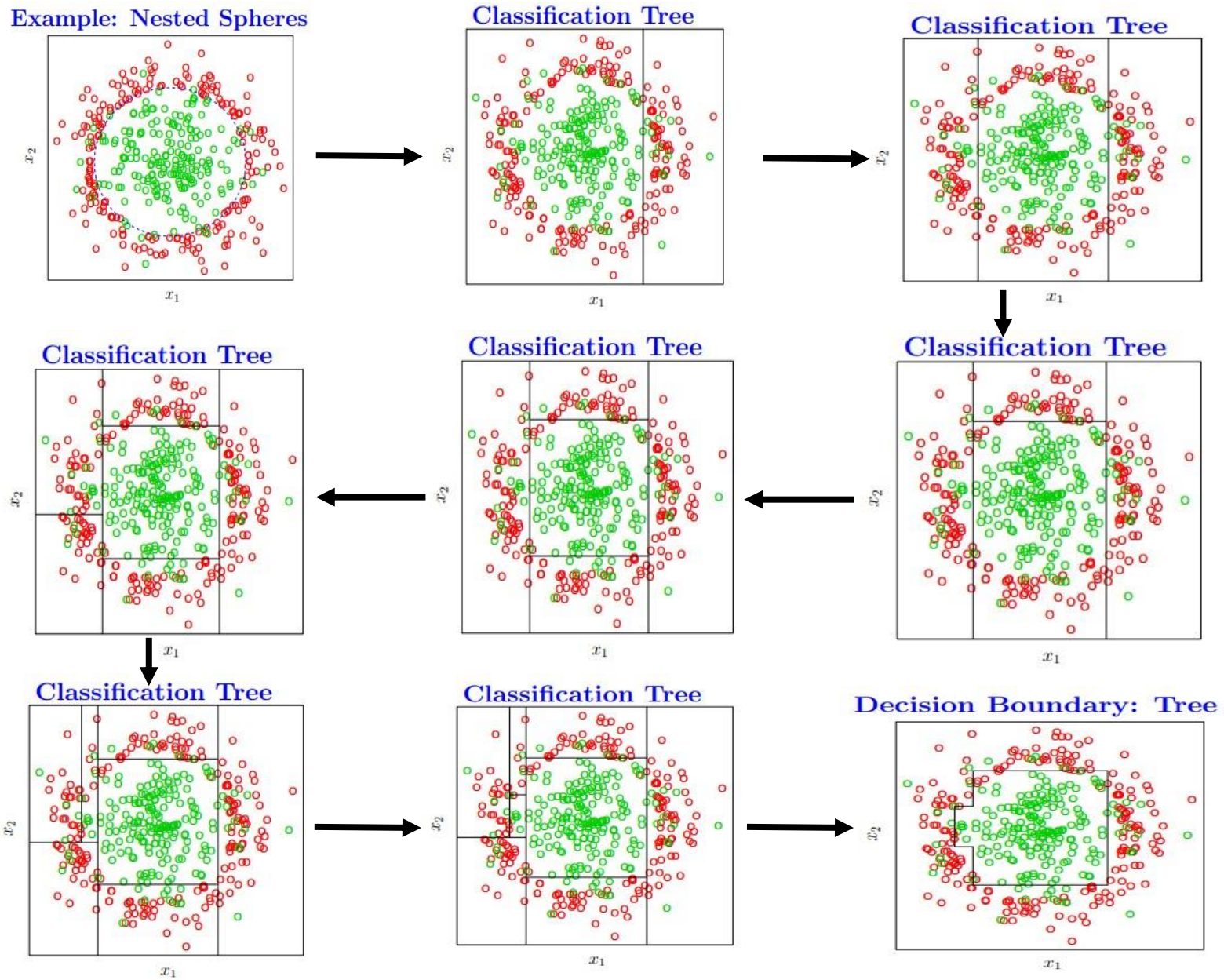


三种决策树学习算法总结

- 适应信息增益来进行特征选择的决策树学习过程，即为ID3决策。
- 所以如果是取值更多的属性，更容易使得数据更“纯”，其信息增益更大，决策树会首先挑选这个属性作为树的顶点。结果训练出来的形状是一棵庞大且深度很浅的树，这样的划分是极为不合理的。
- C4.5：信息增益率 $g_r(D,A) = g(D,A) / H(A)$
- CART：基尼指数
- 总结：一个属性的信息增益越大，表明属性对样本的熵减少的能力更强，这个属性使得数据由不确定性变成确定性的能力越强。

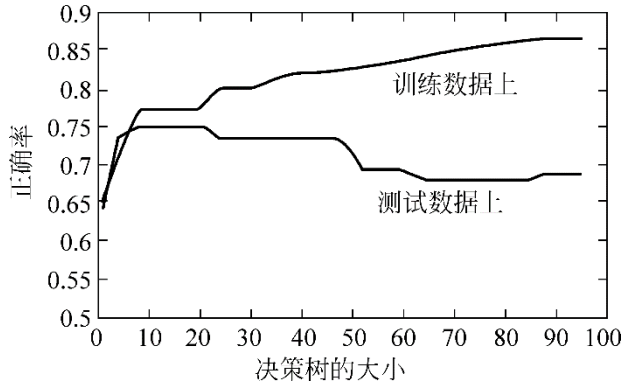
例子：对于下面的数据，希望分割成红色和绿色两个类

决策树生成过程



过学习与决策树的剪枝

- 过学习问题：
  - 算法在训练数据上表现好，在测试数据或新样本上表现较差：



- 剪枝
  - 先剪枝：
    - 数据划分法
    - 阈值法
    - 信息增益的统计显著性分析
  - 后剪枝：
    - 减少分类错误修剪法
    - 最小代价与复杂性的折衷
    - 最小描述长度（MDL）准则

4.11 组合分类器

集成学习(Ensemble learning)是机器学习中近年来的一大热门领域。其中的集成方法是用多种学习方法的组合来获取比原方法更优的结果

用于组合的算法是弱学习算法，即分类正确率仅比随机猜测略高的学习算法，但是组合之后的效果高于强学习算法，即集成之后的算法准确率和效率都很高

（一）装袋法(Bagging)

通过对数据进行**有放回的抽样**，生成多个不同的训练集，并分别训练多个分类器。最终结果通常通过投票或平均的方法**结合各个分类器的预测**。随机森林就是基于Bagging的典型算法

特点：Bagging通过减少模型的方差（variance）来提高稳定性和准确性，尤其适合高方差模型（如决策树）。Bagging还可以**降低过拟合的风险**，因为它结合了多个模型的预测，**减少了单一模型的偏差**

代表算法：随机森林（Random Forest）

（二）提升法(Boosting)

Boosting方法通过逐步生成一系列的弱分类器，每一步都会重点关注前一轮中分类错误的数据。每个弱分类器的权重由其在前一轮中的表现决定，最终的模型是多个弱分类器的加权组合

特点：Boosting通过**减少模型的偏差**（bias）来提升模型的预测能力，尤其适用于偏差大的模型。Boosting在分类和回归任务中均表现出色，能够取得较高的准确率，但**容易过拟合**

代表算法：AdaBoost

（三）梯度提升决策树算法（GBDT）

GBDT是Boosting的改进版本，通过在每一步生成一个新的决策树来减少当前模型的误差，从而逐步优化损失函数。GBDT在每一步优化梯度方向，以更好地逼近目标

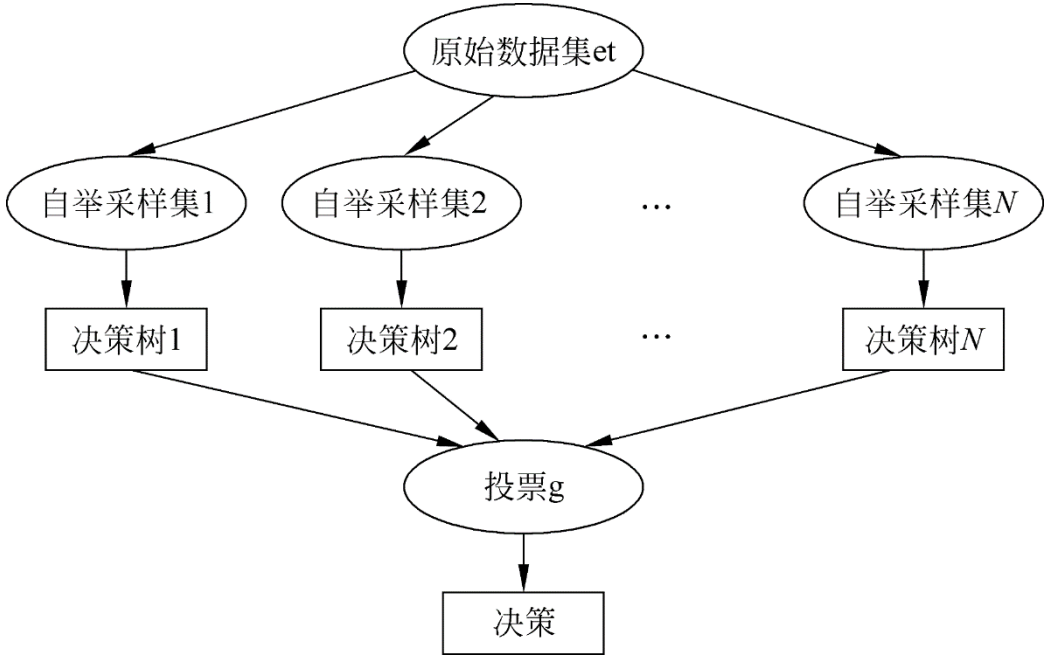
代表算法：**XGBoost**、LightGBM、CatBoost等基于GBDT的变种

（四）随机森林

随机森林结合了Bagging和决策树，通过对不同特征子集和数据子集上的决策树进行训练，最终的预测结果由多个树的投票或平均生成

特点：随机森林具有良好的**鲁棒性和稳定性**，对**数据噪声不敏感**，能够很好地处理高维数据集和缺失值。相比单独的决策树，随机森林具有更强的抗过拟合能力和更高的准确性

随机森林通过多个树的投票或平均增强了模型的泛化能力，适合大多数分类和回归问题



4.12 类别不平衡问题

如果不同类别的训练样例数目稍有差别，通常影响不大，但若差别很大，则会对学习过程造成困扰。例如有998个反例，但是正例只有2个，那么学习方法只需要返回一个永远将新样本预测为反例的学习器，就能达到99.8%的精度；然而这样的学习器往往没有价值，因为它不能预测出任何正例

类别不平衡（class-imbalance）就是指分类任务中不同类别的训练样例数目差别很大的情况

类别不平衡问题的解决——数据分级法

（一）欠采样

直接对训练集中多数类样本进行“欠采样”(undersampling)，即去除一些多数类中的样本使得正例、反例数目接近，然后再进行学习

欠采样代表性算法：EasyEnsemble、BalanceCascade

类别不平衡问题的解决——数据分级法

（二）过采样

对训练集里的少数类进行“过采样”(oversampling)，即增加一些少数类样本使得正、反例数目接近，然后再进行学习

过采样代表性算法：SMOTE

类别不平衡问题的解决——代价敏感学习(cost-sensitive learning)

在传统的机器学习分类问题中，通常假设所有错误的代价是相同的。然而在实际应用中，错误的代价可能不均衡，比如在医疗诊断中，将病人误诊为健康比将健康人误诊为病人的代价要大得多

而为了解决类别不平衡问题，代价敏感学习是在原始标准代价损失函数的基础上，增加了一些约束和权重条件，使得最终代价的数值计算**朝向增加小样本的权重的方向偏置**

讨论

- 非线性 vs. 线性
- 特定非线性方法 vs. 通用非线性方法
- BP与径向基函数网络
- 支持向量机
- 决策树
- 组合分类器、集成学习
- 类别不平衡问题